

题目 1(正则表达式与自动机)

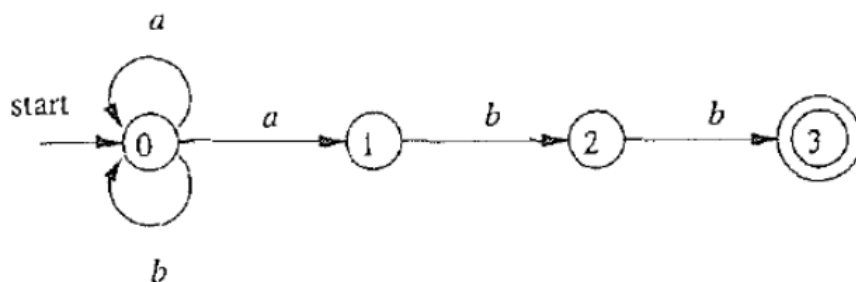
- Thompson 构造法 (从正则表达式到 NFA)
- 子集构造法 (NFA 到 DFA)
- DFA 最小化
- 参考课程录屏 4-automata-20230315

概念:

1. NFA——非确定性有穷自动机 *Nondeterministic Finite Automaton*

- 非确定性有穷自动机 A 是一个五元组 $A = (\Sigma, S, s_0, \delta, F)$:

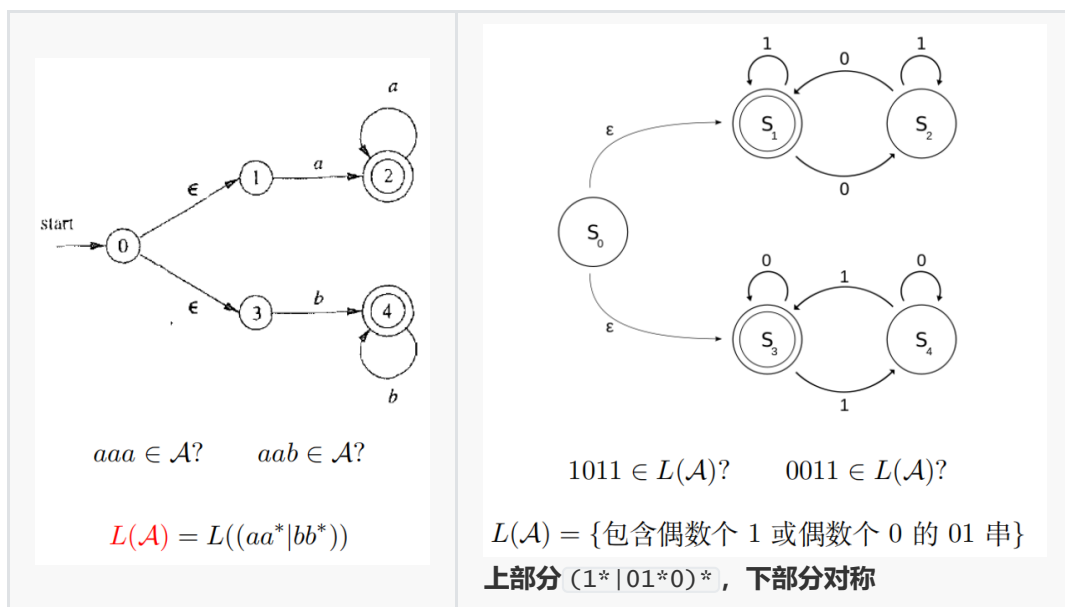
1. 字母表 $\Sigma (\epsilon \notin \Sigma)$: 转移边上的字母
2. **有穷**的状态集合 S : 下图的圆圈
3. **唯一**的初始状态 $s_0 \in S$: 一个没有来源的箭头指向初始状态
4. 状态转移函数 δ : 当前处在某一个状态, 看到一个字母或 ϵ , 会转移到 S 的幂集, 数学语言如下
$$\delta: S \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^S$$
5. 接受状态集合 $F \subseteq S$: 接受状态可以有多个, 双圈来表示



- 什么是“非确定性”?
 - 如上图, 初始状态接受一个 a , 可能转移到 1 状态, 也可能转移到 0 状态
 - 另外, 看到 ϵ (即不看到任何字符), 也可以自发跑到任何状态上去, 但需要有这个状态转移函数
- 关于状态转移函数有一个约定, 没有对应出边的字符默认指向一个不存在的空状态, 比如上图下表

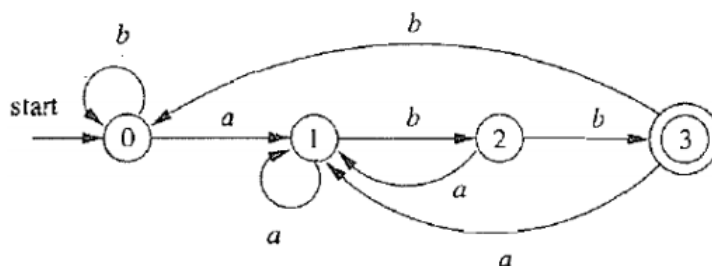
状态	a	b	ϵ
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

- 一些例子



2. DFA——确定性有穷自动机

- 确定性有穷自动机 A 是一个五元组 $A = (\Sigma, S, s_0, \delta, F)$:
 - 字母表 $\Sigma (\epsilon \notin \Sigma)$
 - 有穷**的状态集合 S
 - 唯一**的初始状态 $s_0 \in S$
 - 状态转移函数 δ : 转移到下一个状态, 必须要消耗一个字符 (不能 ϵ), 且下一状态唯一
 $\delta: S \times \Sigma \rightarrow S$
 - 接受状态集合 $F \subseteq S$



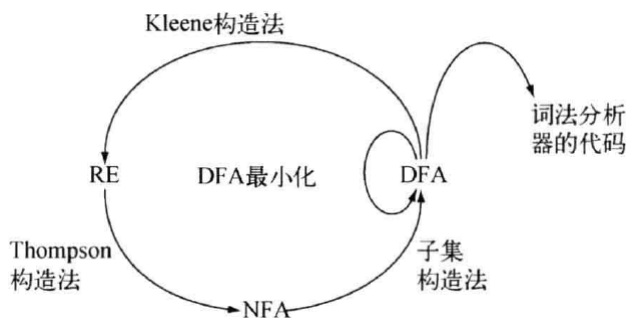
可以发现这张图和上面NFA的例子表示的 $L(A)$ 是一样的

- 如何消除“非确定性”?
 - NFA可以消耗 ϵ 转移到下一状态, 而DFA必须要消耗一个字符
 - NFA的接受一个字符可以转移到多个状态 (幂集), DFA的下一状态唯一
 - 可以通过比较转移状态函数得到:
 - NFA: $\delta: S \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^S$
 - DFA: $\delta: S \times \Sigma \rightarrow S$
- 约定: 所有没有对应出边的字符默认指向一个“死状态”
 - 死状态不管接受什么字符, 都回到自身, 而其自身非接受状态, 所以这个字符串不能被接受

3. 对比 NFA vs. DFA

- NFA简洁易于理解, 便于描述语言 $L(A)$
- DFA没有不确定, 易于判断 $x \in L(A)$, 适合产生词法分析器
- 用NFA描述语言, 用DFA实现词法分析器
- RE $\Rightarrow$ NFA $\Rightarrow$ DFA $\Rightarrow$ 词法分析器

算法（重点部分）



1. 从 RE 到 词法分析器：Thompson构造法

- 将正则表达式 r 通过Thompson构造法变为NFA $N(r)$
 - $r \rightarrow N(r)$
 - 要求 $L(N(r)) = L(r)$, 自动机与正则表达式接收的语言为同一种语言
- Thompson构造法的基本思想：**按结构归纳**
- 回顾正则表达式定义，对每一种定义，都对应给出构造方法即可

Definition (正则表达式)

给定字母表 Σ , Σ 上的正则表达式**由且仅由**以下规则定义：

- ϵ 是正则表达式；
- $\forall a \in \Sigma, a$ 是正则表达式；
- 如果 s 是正则表达式，则 (s) 是正则表达式；
- 如果 s 与 t 是正则表达式，则 $s|t, st, s^*$ 也是正则表达式。

- ϵ 是正则表达式



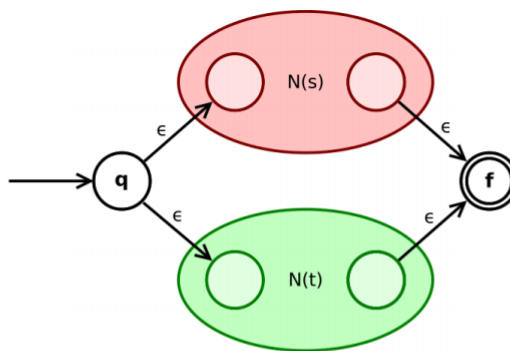
- $a \in \Sigma$ 是正则表达式



- 如果 s 是正则表达式，则 (s) 是正则表达式

$$N((s)) = N(s).$$

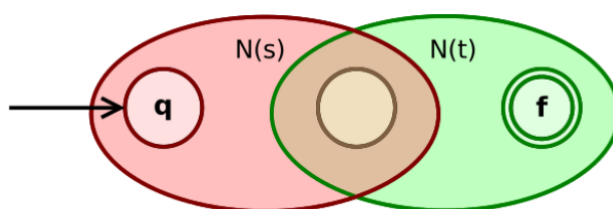
1. 如果 s, t 是正则表达式，则 $s|t$ 是正则表达式



Q: 如果 $N(s)$ 或 $N(t)$ 的开始状态或接受状态不唯一, 怎么办?

根据归纳假设, $N(s)$ 与 $N(t)$ 的开始状态与接受状态均唯一。

2. 如果 s, t 是正则表达式, 则 st 是正则表达式

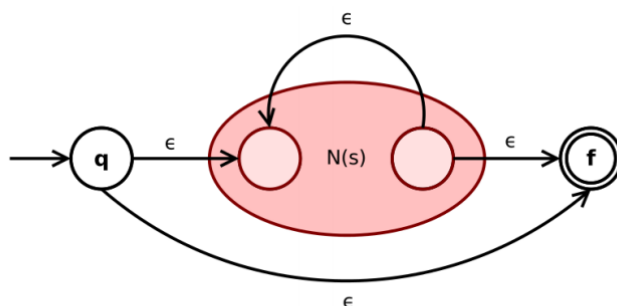


根据归纳假设, $N(s)$ 与 $N(t)$ 的开始状态与接受状态均唯一。

终止状态和开始状态重合

我们的算法是递归构造的, 前两种基本情况都为单输入单输出, 所以在此基础上构造也必然为单输入单输出

3. 如果 s, t 是正则表达式, 则 s^* 是正则表达式



根据归纳假设, $N(s)$ 的开始状态与接受状态唯一。

• $N(r)$ 的性质以及Thompson构造法复杂度分析

1. $N(r)$ 的开始状态与接受状态均唯一
2. 开始状态没有入边, 接受状态没有出边
3. $N(r)$ 的状态数 $|S| \leq 2 \times |r|$ 。 ($|r|$: r 中运算符与运算分量的总和, 也就是进行了多少次构造)

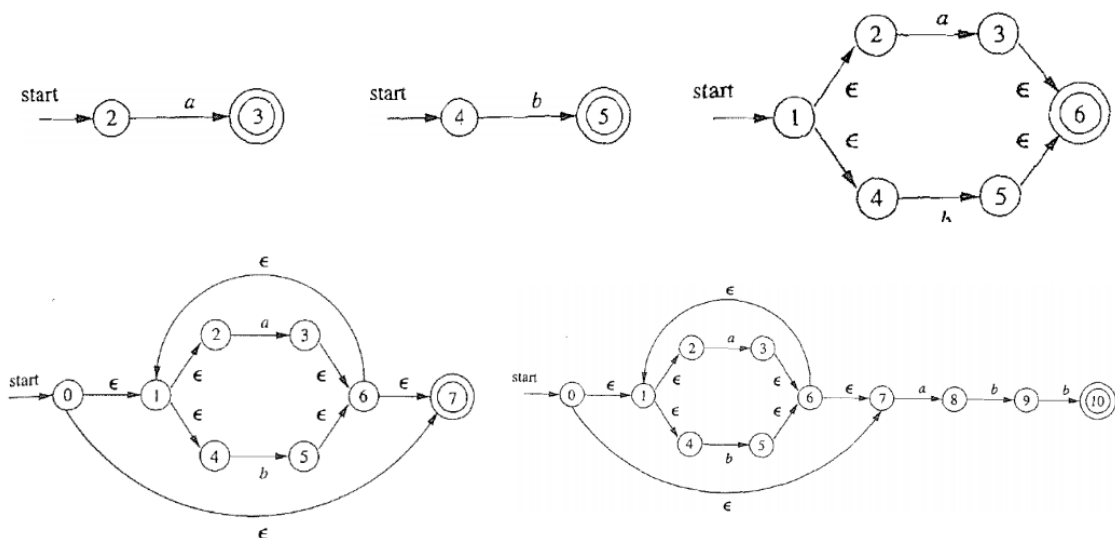
每一次构造最多有两个状态变更: 一个输入和一个输出

4. 每个状态最多有两个 ϵ -入边与两个 ϵ -出边。

5. $\forall a \in \Sigma$, 每个状态最多有一个 a -入边与一个 a -出边

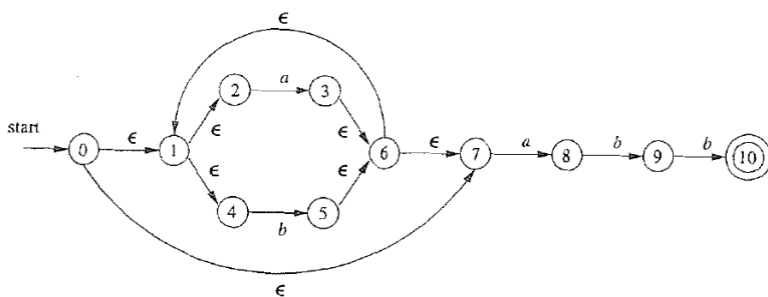
- 一个例子 (考试: 大题第一问)

$$r = (a|b)^*abb$$

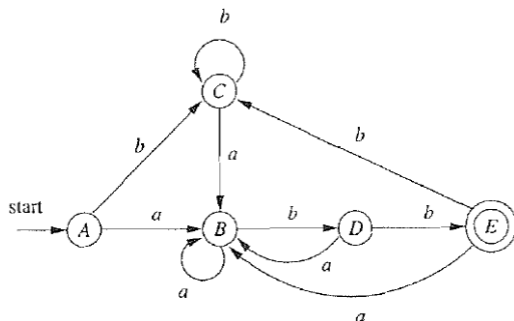


2. 从NFA到 DFA: 子集构造法

- 要求: $L(N) = L(D)$, 即NFA与DFA接受的语言是同一种语言
- 子集构造法思想: **用DFA模拟NFA**
 - (或称为幂集构造法, *Subset/Powerset Construction*)
 - 示例:



$$L(\mathcal{A}) = L((a|b)^*abb)$$



- 过程: DFA就两个部分 (状态集合和转态转移关系)
 - 先确定初始状态, 看初始状态在字符驱动下会到哪些新的状态中
 - 如此不断扩展DFA, 直到无法扩展
 - 以上例

1. 初始状态 (start点和通过 ϵ 可到达的状态)

$\{0, 1, 2, 4, 7\} \Rightarrow$ DFA整体的初始状态, 记为A

2. 初始状态在 a, b 的驱动下, 初始状态中每一个状态会到那个状态上去

不要忽视 ϵ 的作用

在 a 的驱动下: $\{3, 8, 6, 7, 1, 2, 4\}$, 记为B

在 b 的驱动下: $\{5, 6, 7, 1, 2, 4\}$, 记为C

3. 不断重复这个过程来扩展DFA

■ 哪些状态是接受状态?

■ 包含NFA中某一个接受状态的DFA状态

NFA 状态	DFA 状态	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8\}$	B	B	D
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 4, 5, 6, 7, 9\}$	D	B	E
$\{1, 2, 4, 5, 6, 7, 10\}$	E	B	C

○ 算法涉及的几个运算

1. ϵ -闭包: 从状态 s 开始, 只通过 ϵ -转移可达的状态集合

$$\epsilon\text{-closure}(s) = \{t \in S_N \mid s \xrightarrow{\epsilon^*} t\}$$

$$\epsilon\text{-closure}(T) = \bigcup_{s \in T} \epsilon\text{-closure}(s)$$

$$2. \text{move}(T, a) = \bigcup_{s \in T} \delta(s, a)$$

○ 子集构造法 ($N \Rightarrow D$) 的原理 (符号化):

$$N : (\Sigma_N, S_N, n_0, \delta_N, F_N)$$

$$D : (\Sigma_D, S_D, d_0, \delta_D, F_D)$$

$$\Sigma_D = \Sigma_N$$

$$S_D \subseteq 2^{S_N} \quad (\forall s_D \in S_D : s_D \subseteq S_N)$$

$$\text{初始状态 } d_0 = \epsilon\text{-closure}(n_0)$$

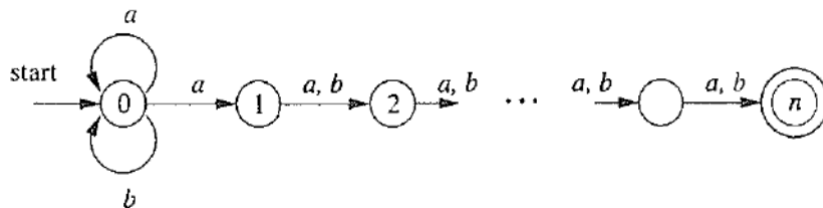
$$\text{转移函数 } \forall a \in \Sigma_D : \delta_D(s_D, a) = \epsilon\text{-closure}(\text{move}(s_D, a))$$

$$\text{接受状态集 } F_D = \{s_D \in S_D \mid \exists f \in F_N. f \in s_D\}$$

• 一些例子

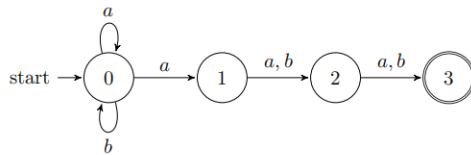
1. 长度为 $m \geq n$ 个字符的 a, b 串, 且倒数第 n 个字符是 a

$$L_n = (a|b)^* a (a|b)^{n-1}$$



$L_3 = (a|b)^* a(a|b)(a|b)$

NFA 如下:



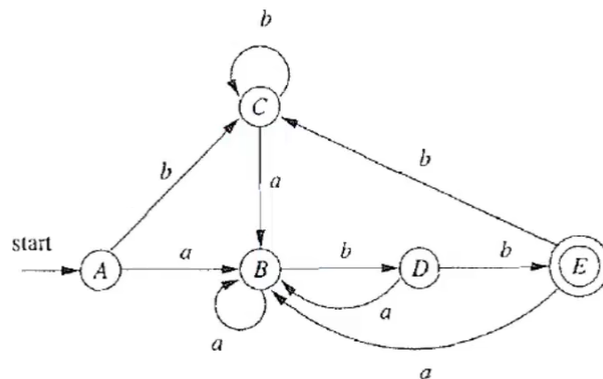
$\epsilon - \text{closure}(0) = \{0\}$
 $\epsilon - \text{closure}(\text{move}(A, a)) = \{0, 1\}$
 $\epsilon - \text{closure}(\text{move}(B, a)) = \{0, 1, 2\}$
 $\epsilon - \text{closure}(\text{move}(B, b)) = \{0, 2\}$
 $\epsilon - \text{closure}(\text{move}(C, a)) = \{0, 1, 2, 3\}$
 $\epsilon - \text{closure}(\text{move}(C, b)) = \{0, 2, 3\}$
 $\epsilon - \text{closure}(\text{move}(D, a)) = \{0, 1, 3\}$
 $\epsilon - \text{closure}(\text{move}(D, b)) = \{0, 3\}$

NFA 状态	DFA 状态	a	b
0	A	B	A
0,1	B	C	D
0,1,2	C	E	F
0,2	D	G	H
0,1,2,3	E	E	F
0,2,3	F	G	H
0,1,3	G	C	D
0,3	H	B	A

- 关于闭包的更多
 - $f\text{-closure}(T)$
 - 闭包是在初始集合上进行的, 做什么闭包是由 f 决定的
 - $T \Rightarrow f(T) \Rightarrow f(f(T)) \Rightarrow f(f(f(T))) \Rightarrow \dots$, 直到找到 x , 使得 $f(x) = x$, x 称为 f 的不动点

3. DFA最小化

- 从NFA通过子集构造法得到的DFA可以进行最小化
- DFA最小化算法基本思想: 等价的状态可以合并**
 - 如何定义等价状态?
 - 看行为是否等价: 看两个状态在任何相同字符驱动下是否会到相同状态上
 - 形式化表示: $s \sim t \Leftrightarrow \forall a \in \Sigma. ((s \xrightarrow{a} s') \wedge (t \xrightarrow{a} t')) \Rightarrow (s' = t')$.
 - 但是这个定义是错误的!!** (按这个定义, A、C、E是等价的, 可恶)



$$A \sim C \sim E$$

- 原因在于: 接受状态和非接受状态必定不等价
 - 修正: 两个状态在任何相同字符驱动下会到等价的状态上**
 - 形式化表示: $s \sim t \Leftrightarrow \forall a \in \Sigma. ((s \xrightarrow{a} s') \wedge (t \xrightarrow{a} t')) \Rightarrow (s' \sim t')$.
- 操作:
 - 基于修正后的定义, 不断合并等价的状态, 直到无法合并为止
 - 但同时给操作带来了问题: 递归定义, 从何入手?

- 解决这个问题，John Hopcroft从反面入手

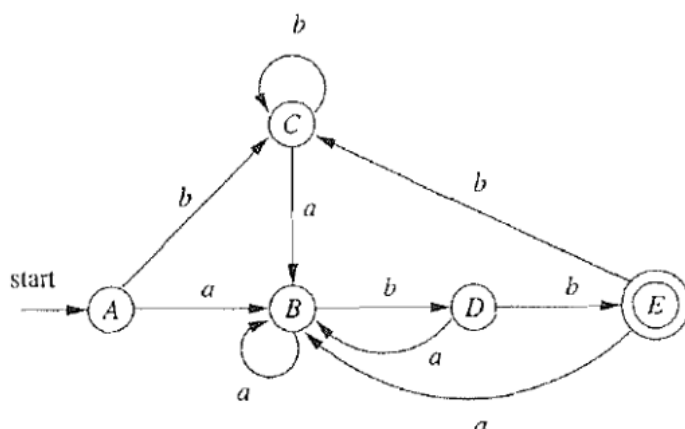
$$s \approx t \Leftrightarrow \exists a \in \Sigma. ((s \xrightarrow{a} s') \wedge (t \xrightarrow{a} t')) \wedge (s' \approx t').$$

即两个状态不等价就意味着存在一个字符驱动它们跑到不同状态上去

- 为什么反着来就能操作了？怎么操作？

- 我们不是做合并，是做分裂
- 我们知道接受状态和非接受状态必定不等价，就从这里开始！
- 再对每一类中，接受相同字符驱动，是不是**每一个成员都跑到同一个大类中去**，如果是则等价，不是则不等价，进行分裂
- 直到无法再分裂位置

○ 例子：



1. 初始按是否为接受状态划分为两类

■ $\{A, B, C, D\}$ 、 $\{E\}$

2. 进行第一次的分裂（这里因为只有一个接受状态，所以这一步只要考虑一组）

	在 a 的驱动下	在 b 的驱动下
A	B	C
B	B	D
C	B	C
D	B	E（与其他不在一个组）
结论	在 a 的驱动下到同一个组，等价	在 b 的驱动下，D与其他不等价

■ 经过这一次分裂，更新为： $\{A, B, C\}$ 、 $\{D\}$ 、 $\{E\}$

3. 再进行分裂（ABC需要再处理一遍，因为D的分裂对他们也会产生影响）

	在 a 的驱动下	在 b 的驱动下
A	B	C
B	B	D（与其他不在一个组）
C	B	C
结论	在 a 的驱动下到同一个组，等价	在 b 的驱动下，B与其他不等价

经过这一次分裂，更新为： $\{A, C\}$ 、 $\{B\}$ 、 $\{D\}$ 、 $\{E\}$

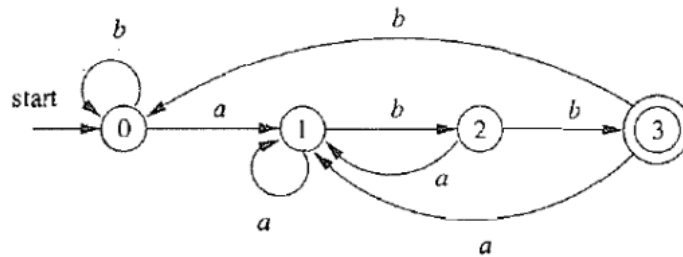
4. 再继续

	在 a 的驱动下	在 b 的驱动下
A	B	C
C	B	C
结论	在 a 的驱动下到同一组，等价	在 b 的驱动下到同一个组，等价

5. 得到结果

$\{A, C\}$ 、 $\{B\}$ 、 $\{D\}$ 、 $\{E\}$

6. 合并等价状态 $A \sim C$



- 注意：划分之前要补充“死状态”，使其是一个DFA，再使用DFA最小化算法（死状态有自环，不要忘记）
- 如果将最小化应用于生成词法分析器的环节，则由于所有接受状态的任务时不一样的，那么一开始分裂的时候就是可以认为他们是不等价的，将他们分为不同类

例题：

考虑正则表达式 $r = a(b|c)^*$ 。①

- (1) 使用 Thompson 构造法构造等价的 NFA; ②
- (2) 使用子集构造法构造等价的 DFA;
- (3) 将上一步构造的 DFA 最小化;
- (4) 将上一步得到的最小 DFA 转化为等价的正则表达式, 记为 r' 。
- (5) r' 与 r 相同吗? 如果不同, 请将 r' 化简为 r 。

以上各小题, 请给出关键的中间步骤。

(不必给出所有的细节, 类似的步骤可以“跳步”; 尽量将解答部分控制在两页以内。)

(4)

step1:

$$R_{00}^{-1} = \epsilon, R_{11}^{-1} = b|c|\epsilon, R_{01}^{-1} = a, R_{10}^{-1} = \emptyset$$

step2:

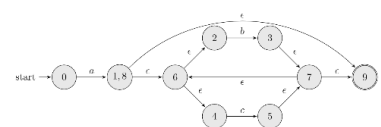
$$R_{01}^0 = R_{00}^{-1}(R_{00}^{-1})^*R_{01}^{-1} = a, R_{11}^0 = R_{10}^{-1}(R_{00}^{-1})^*R_{01}^{-1} = \emptyset(b|c|\epsilon) = b|c|\epsilon$$

step3:

$$R_{01}^1 = R_{01}^0(R_{01}^0)^*R_{01}^0 = (a(b|c|\epsilon)^*(b|c|\epsilon))|a = a(b|c|\epsilon)^*(b|c|\epsilon) = a(b|c)^*$$

(5)

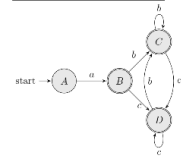
相同



(2)

状态 1,8 用 1 表示

NFA	DFA	a	b	c
{0}	A	B	B	B
{1,2,4,6,9}	B	B	C	D
{2,3,4,6,7,9}	C	B	C	D
{2,4,5,6,7,9}	D	B	C	D

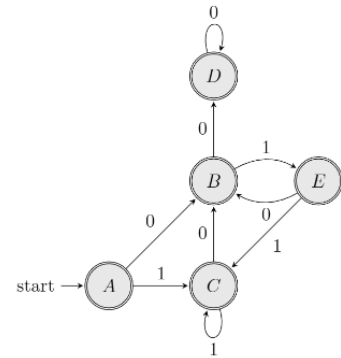


(3)

初始划分 $\{A, \{B, C, D\}\}$, 且 $\{B, C, D\}$ 对于输入 a,b,c 有相同转移, 不可分, 则最终划分为 $\{A, \{B, C, D\}\}$ 。



NFA 状态	DFA 状态	0	1
0,1,2,3,8,9,11	A	B	C
5,9,10,11	B	D	E
1,2,3,4,7,8,9,11	C	B	C
9,10,11	D	D	∅
1,2,3,6,7,8,9,11	E	B	C



(3)

$$\Pi_0 = \{\{A, B, C, D, E\}\}$$

$$\Pi_1 = \{\{A, B, C, E\}, \{D\}\}$$

$$\Pi_2 = \{\{A, C, E\}, \{B\}, \{D\}\}$$

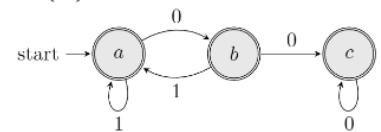
$$\Pi_3 = \Pi_2 = \Pi_{final}$$

或者直接看出来合并 A,C,E

$$a = \{A, C, E\}$$

$$b = \{B\}$$

$$c = \{D\}$$



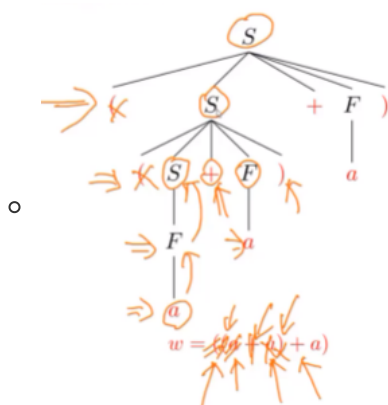
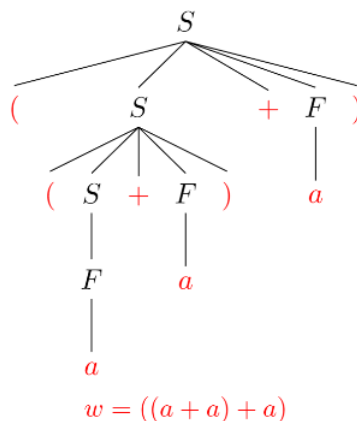
题目 2(LL语法分析)

- First 集合与 Follow 集合
- 构造预测分析表
- 参考课程录屏 7-parser-ll-20230331、7-parser-ll-8-parser-allstar-20230407

概念

- 自顶向下、递归下降、基于预测表的
- LL的含义（自顶向下的构建语法分析树，从根节点不断展开）
 - 第一个L：从左到右读入词法单元
 - 第二个L：总是选择最左边的非终结符进行展开
- 递归下降——为每一个非终结符写一个递归函数展开

$S \rightarrow F$
$S \rightarrow (S + F)$
$F \rightarrow a$



- 为何同样是展开非终结符 S ，前两次选择了 $S \rightarrow (S + F)$ ，而第三次选择了 $S \rightarrow F$ ？
 - 因为它们面对的当前词法单元不同
 - 关键问题引出——如何选择产生式——适用预测分析表

- 适用预测分析表确定产生式

	(	)	a	+	$\$$
S	2		1		
F			3		

指明了每个非终结符在面对不同词法单元或文件结构结束符时，该选择哪个产生式（按编号进行索引）或者报错（空单元格）

- LL(1)文法
 - 如果文法 G 的预测分析表是**无冲突**的，则 G 是 $LL(1)$ 文法
 - **无冲突**：每个单元格里只有一个产生式的编号
 - 对于**当前**选择的非终结符，进根据输入中的**当前词法单元**（LL(1)）即可确定需要使用哪条产生式

算法

- 递归下降、预测分析的实现方法

$S \rightarrow F$
 $S \rightarrow (S + F)$
 $F \rightarrow a$

	(	)	a	+	\$
S	2		1		
F			3		

```

1: procedure MATCH(t)
2:   if token = t then
3:     token ← NEXT-TOKEN()
4:   else
5:     ERROR(token, t)

```

```

1: procedure F()
2:   if token = 'a' then
3:     MATCH('a')
4:   else
5:     ERROR(token, {'a'})

```

```

1: procedure S()
2:   if token = '(' then
3:     MATCH('(')
4:     S()
5:     MATCH('+')
6:     F()
7:     MATCH(')')
8:   else if token = 'a' then
9:     F()
10:  else
11:    ERROR(token, {'(', 'a'})

```

1. First集合

- $First(\alpha)$ 是可以从 α 推导得到的句型的**首终结符号** (可能有 ϵ) 的集合
- 对于任意的 (产生式的右部) $\alpha \in (N \cup T)^*$:

$$FIRST(\alpha) = \{t \in T \cup \{\epsilon\} \mid \alpha \xRightarrow{*} t\beta \vee \alpha \xRightarrow{*} \epsilon\}.$$

考虑非终结符 A 的所有产生式 $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_m$,

如果它们对应的 $FIRST(\alpha_i)$ 集合互不相交,

- 则只需查看当前输入词法单元, 即可确定选择哪个产生式 (或报错)
- 计算方法:

1. 先计算每个符号 X 的 $First(X)$ 集合

- X 是终结符, $First(X) = \{X\}$
- $X \rightarrow Y_1 Y_2 \dots Y_K$
 - 先把 $First(Y_1) \setminus \{\epsilon\}$ 并入 $First(X)$ 中
 - 如果 Y_1 能推导出 ϵ , 则要将 $First(Y_2) \setminus \{\epsilon\}$ 并入 $First(X)$ 中
 - 依次类推
 - 如果 Y_1 到 Y_k 都能推出 ϵ , 则说明 X 能推出 ϵ , 则把 ϵ 并入 $First(X)$ 中

```

1: procedure FIRST( $X$ )
2:   if  $X \in T$  then                                ▷ 规则 1:  $X$  是终结符
3:     FIRST( $X$ ) =  $X$                                      ↳
4:   for  $X \rightarrow Y_1 Y_2 \dots Y_k$  do                    ▷ 规则 2:  $X$  是非终结符
5:     FIRST( $X$ )  $\leftarrow$  FIRST( $X$ )  $\cup$  {FIRST( $Y_1$ )  $\setminus$  { $\epsilon$ }}
6:     for  $i \leftarrow 2$  to  $k$  do
7:       if  $\epsilon \in L(Y_1 \dots Y_{i-1})$  then
8:         FIRST( $X$ )  $\leftarrow$  FIRST( $X$ )  $\cup$  {FIRST( $Y_i$ )  $\setminus$  { $\epsilon$ }}
9:       if  $\epsilon \in L(Y_1 \dots Y_k)$  then                ▷ 规则 3:  $X$  可推导出空串
10:        FIRST( $X$ )  $\leftarrow$  FIRST( $X$ )  $\cup$  { $\epsilon$ }

```

■ 不断应用上面的规则, 直到每个 FIRST(X) 都不再变化 (不动点!!!)

2. 再计算每个符号串 α 的 $First(\alpha)$ 集合

$$\alpha = X\beta$$

$$FIRST(\alpha) = \begin{cases} FIRST(X) & \epsilon \notin L(X) \\ (FIRST(X) \setminus \{\epsilon\}) \cup FIRST(\beta) & \epsilon \in L(X) \end{cases}$$

最后, 如果 $\epsilon \in L(\alpha)$, 则将 ϵ 加入 FIRST(α)。

• 例子:

(1) $X \rightarrow Y$	
(2) $X \rightarrow a$	
(3) $Y \rightarrow \epsilon$	$FIRST(X) = \{a, c, \epsilon\}$
(4) $Y \rightarrow c$	$FIRST(Y) = \{c, \epsilon\}$
(5) $Z \rightarrow d$	$FIRST(Z) = \{a, c, d\}$
(6) $Z \rightarrow XYZ$	$FIRST(XYZ) = \{a, c, d\}$

先算 $First(X)$ 、 $First(Y)$ 、 $First(Z)$

- $First(X) = \{a, First(Y)\}$
- $First(Y) = \{\epsilon, c\}$
 - $First(X) = \{a, c, \epsilon\}$
- $First(Z) = \{d, a, c, First(Z)\}$
 - $First(Z) \subseteq First(Z)$, 即没变化
- 整个 $Z \rightarrow XYZ$ 能不能推出 ϵ ? 不能
- 所以 $First(Z) = \{d, a, c\}$

再算字符串 XYZ 的 $First(XYZ)$ 集合

- $First(XYZ) = \{a, c, \}$ // $L(X)$ 里有 ϵ , 所以先不放进去
- $First(XYZ) = \{a, c, \}$ // $L(Y)$ 里有 ϵ , 所以先不放进去
- $First(XYZ) = \{a, c, d\}$

2. Follow集合

- $Follow(A)$ 是可能在某些句型中紧跟在 A 右边的终结符 (可能是 $\$$) 的集合
- 对于任意的 (产生式的左部) 非终结符 $A \in N$:

$$FOLLOW(A) = \{t \in T \cup \{\$\} \mid \exists s. S \xRightarrow{*} s \triangleq \beta A t \gamma\}.$$

考虑产生式 $A \rightarrow \alpha$,

如果从 α 可能推导出空串 ($\alpha \xRightarrow{*} \epsilon$),

- 则只有当当前词法单元 $t \in FOLLOW(A)$, 才可以选择该产生式
- 计算方法: 为每个非终结符 X 计算 $Follow(X)$ 集合
 - X 是开始符号 (没有指出则为第一个产生式的左部) , $Follow(X) \leftarrow Follow(X) \cup \{\$\}$
 - X 是某产生式右部的最后一个符号 ($A \rightarrow \alpha X$) ,
 $Follow(X) \leftarrow Follow(X) \cup Follow(A)$
 - 所有可能跟在 A 后面的都有可能跟在 X 后面
 - X 是某产生式右部中间的一个符号 ($A \rightarrow \alpha X \beta$)
 - $Follow(X) \leftarrow Follow(X) \cup (First(\beta) \setminus \{\epsilon\})$
 - 如果 $\epsilon \in First(\beta)$, 则 $Follow(X) \leftarrow Follow(X) \cup Follow(A)$

```
1: procedure FOLLOW( $X$ )
2:   for  $X$  是开始符号 do           ▷ 规则 1:  $X$  是开始符号
3:     FOLLOW( $X$ )  $\leftarrow$  FOLLOW( $X$ )  $\cup$   $\{\$\}$ 
4:   for  $A \rightarrow \alpha X$  do           ▷ 规则 2:  $X$  是某产生式右部的最后一个符号
5:     FOLLOW( $X$ )  $\leftarrow$  FOLLOW( $X$ )  $\cup$  FOLLOW( $A$ )
6:   for  $A \rightarrow \alpha X \beta$  do       ▷ 规则 3:  $X$  是某产生式右部中间的一个符号
7:     FOLLOW( $X$ )  $\leftarrow$  FOLLOW( $X$ )  $\cup$  (FIRST( $\beta$ )  $\setminus$   $\{\epsilon\}$ )
8:     if  $\epsilon \in FIRST(\beta)$  then
9:       FOLLOW( $X$ )  $\leftarrow$  FOLLOW( $X$ )  $\cup$  FOLLOW( $A$ )
```

- 不断应用上面的规则, 直到每个 $FOLLOW(X)$ 都不再变化 (不动点!!!)
- 例子:

(1) $X \rightarrow Y$

(2) $X \rightarrow a$

(3) $Y \rightarrow \epsilon$

(4) $Y \rightarrow c$

(5) $Z \rightarrow d$

(6) $Z \rightarrow XYZ$

$FOLLOW(X) = \{a, c, d, \$\}$

$FOLLOW(Y) = \{a, c, d, \$\}$

$FOLLOW(Z) = \emptyset$

- $Follow(X) = \{\$, \}$ // X 是一个开始符号, 出现在左边
 - 然后看产生式的右部, 发现 $Z \rightarrow XYZ$ 中包含 X , X 在产生式右部中间, 应用第三条规则, 因为 $First(YZ) = \{a, c, d\}$, 所以
 $Follow(X) = \{\$, a, c, d\}$
- $Follow(Y) = \{\}$ // Y 不是一个开始符号

- 然后看产生式右部, 发现 $X \rightarrow Y$ 中包含 Y , Y 是产生式右部最后一个符号, 应用第二条规则, 所以 $Follow(X) \subseteq Follow(Y)$,
 $Follow(Y) = \{\$, a, c, d, \}$
- 继续看, 发现 $Z \rightarrow XYZ$ 中包含 Y , Y 在产生式右部中间, 应用第三条规则, 因为 $First(Z) = \{a, c, d\}$, 所以 $Follow(Y) = \{\$, a, c, d\}$
- $Follow(Z) = \{\}$ // Z 不是一个开始符号
- 然后看产生式的右部, 发现 $Z \rightarrow XYZ$ 中包含 Z , Z 是产生式右部最后一个符号, 应用第二条规则, 所以 $Follow(Z) \subseteq Follow(Z)$, 所以 $Follow(Z) = \emptyset$

3. 构造预测分析表

- 每一行为一个非终结符, 每一列是一个终结符 (以及 $\$$)

对应每条产生式 $A \rightarrow \alpha$ 与终结符 t , 如果

$$t \in \text{FIRST}(\alpha) \quad (1)$$

$$\alpha \xRightarrow{*} \epsilon \wedge t \in \text{FOLLOW}(A) \quad (2)$$

则在表格 $[A, t]$ 中填入 $A \rightarrow \alpha$ (编号)。

那个带*的箭头表示: α 经过若干步推导能够推出 ϵ

- 含义:
 - 遍历每一条产生式, 对于任意一个终结符 t , 如果 t 在 $First(\alpha)$ 里面, 则意味着可能可以使用此产生式
 - 另外 A 有可能导出 ϵ (即 A “消失”), 则 t 在 $Follow(A)$ 里面, 才意味着可能使用这个产生式
- 另外一种等价表示

$$t \in \text{FIRST}(\alpha) \quad (1)$$

$$\epsilon \in \text{FIRST}(\alpha) \wedge t \in \text{FOLLOW}(A) \quad (2)$$

当下的选择未必正确, 但 “你别无选择”。

- 例子

(1) $X \rightarrow Y$
(2) $X \rightarrow a$
(3) $Y \rightarrow \epsilon$
(4) $Y \rightarrow c$
(5) $Z \rightarrow d$
(6) $Z \rightarrow XYZ$

$$\text{FIRST}(X) = \{a, c, \epsilon\}$$

$$\text{FIRST}(Y) = \{c, \epsilon\}$$

$$\text{FIRST}(Z) = \{a, c, d\}$$

$$\text{FIRST}(XYZ) = \{a, c, d\}$$

$$\text{FOLLOW}(X) = \{a, c, d, \$\}$$

$$\text{FOLLOW}(Y) = \{a, c, d, \$\}$$

$$\text{FOLLOW}(Z) = \emptyset$$

	a	c	d	$\$$
X	1, 2	1	1	1
Y	3	3, 4	3	3
Z	6	6	5, 6	

- 先根据 $First(\alpha)$ 进行填充, 如果 $First(\alpha)$ 中包含 ϵ , 则把 $Follow(A)$ 中的元素也进行填充
- 判断是否为LL(1)文法: 是否无冲突, 检查是否有一个单元格中有超过一个表达式

5. 不是LL(1)文法——改造

1. 消除左递归

- 为什么左递归在 LL(1) 不行——在不消耗任何词法单元的情况下, 直接递归调用造成死循环

直接左递归	直接左递归 (扩展)	间接左递归
$A \rightarrow A\alpha \beta$	$A \rightarrow A\alpha_1 A\alpha_2 \dots A\alpha_m \beta_1 \beta_2 \dots\beta_n$	$S \rightarrow Ac c$ $A \rightarrow Bb b$ $B \rightarrow Sa a$
$A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \epsilon$	$A \rightarrow \beta_1 A' \beta_2 A' \dots \beta_n A'$ $A' \rightarrow \alpha_1 A' \alpha_2 A' \dots \alpha_m A' \epsilon$	$S \rightarrow Ac c$ $A \rightarrow Bb b$ $B \rightarrow (bca ca a)B'$ $B' \rightarrow bcaB' \epsilon$

- 间接左递归是如何产生的 $S \Rightarrow Ac \Rightarrow Bbc \Rightarrow Sabc$, 绕了一圈回到自己, 为了打破之, 可以给每个非终结符编号 A_i , 并修改文法使之满足 $A_m \rightarrow A_n \alpha \Rightarrow m < n$
- 如上例:

给非终结符编号:

$S : A_1$ 、 $A : A_2$ 、 $B : A_3$

检查每一条产生式:

1. $S \rightarrow Ac|c$, 这里其实是两条
 - 对于 $S \rightarrow Ac$, 即 $(A_1 \rightarrow A_2c)$, 没有问题
 - 对于 $S \rightarrow c$, 没有问题
2. $A \rightarrow Bb|b$, 也是两条, 也都没有问题
3. $B \rightarrow Sa|a$, 也是两条, 发现 $B \rightarrow Sa$, 即 $A_2 \rightarrow A_1a$, 是有问题的, 如何改造?
 - 替换掉 S , 可以理解为这里在做把 S 放大的工作 $B \rightarrow Aca|ca$, 还不够
 - 替换掉 A , $B \rightarrow Aca|ca \rightarrow ((Bb|b)ca)|ca$, 即 $B \rightarrow Bbca|bca|ca$, 得到一个直接左递归, 在用左递归处理即可

算法要求:

文法中不存在环 (形如 $A \xRightarrow{*} A$ 的推导)

文法中不存在 ϵ 产生式 (形如 $A \rightarrow \epsilon$ 的产生式)

$S \rightarrow Aa b$
$A \rightarrow Ac Sb \epsilon$

2. 提取左公因子

- 例子

```

7      /*
8      decl : 'int' ID ';'
9          | 'int' ID '=' ID ';'
10     ;
11     */
12
13     decl : 'int' ID optional_init ';' ;
14     optional_init
15         : '=' ID # Init
16         |         # NoInit
17         ;
18
19     /*
20     decl : 'int' ID ('=' ID)? ';'
21     */

```

- ANTLR4可以处理有左公因子的文法

例题：

给定下述文法 G ,

$$S \rightarrow aAb \quad (1)$$

$$S \rightarrow bAa \quad (2)$$

$$A \rightarrow cS \quad (3)$$

$$A \rightarrow \epsilon \quad (4)$$

- (1) 为该文法计算必要的 FIRST 集合;
- (2) 为该文法计算必要的 FOLLOW 集合;
- (3) 为该文法设计预测分析表;
- (4) 该文法是 $LL(1)$ 文法吗? 请说明理由。
- (5) 如果该文法是 $LL(1)$ 文法, 请给出相应的 $LL(1)$ 语法分析器的伪代码 (可以使用递归下降实现框架, 也可以使用非递归的版本); 如果该文法不是 $LL(1)$ 文法, 请将其改造成 $LL(1)$ 文法 (不必再重复各小题)。

解答:

- (1) $\text{FIRST}(S) = \{a, b\}$, $\text{FIRST}(A) = \{\epsilon, c\}$
- (2) $\text{FOLLOW}(S) = \{\$, a, b\}$, $\text{FOLLOW}(A) = \{a, b\}$

	a	b	c	$\$$
S	$S \rightarrow aAb$	$S \rightarrow bAa$		
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow cS$	

- (4) 是 $LL(1)$ 文法, 预测分析表没有冲突

Algorithm 1 Parser

```

1: procedure PARSE( $w, M$ )  $\triangleright w$  is lexical string,  $M$  is prediction table
2:    $input \leftarrow w\$, t \leftarrow input.first()$   $\triangleright \$$  is end character,  $t \in \{a, b, c, \$\}$ 
3:    $stack \leftarrow S\$, X \leftarrow stack.top()$   $\triangleright$  use stack to simulate,  $X \in \{a, b, c, A, S, \$\}$ 
4:   while  $X \neq \$$  do
5:     if  $X == t$  then  $\triangleright X$  is terminal and equals to  $t$ 
6:        $stack.pop()$ 
7:        $X = stack.top()$ 
8:        $t = input.next()$ 
9:     else if  $X \in \{a, b, c\}$  then  $\triangleright X$  is terminal and  $X \neq t$ 
10:      error()
11:     else if  $M[X, t]$  is empty then  $\triangleright$  error: no Production
12:      error()
13:     else if  $M[X, t] = X \rightarrow \gamma_1 \gamma_2 \dots \gamma_n$  then  $\triangleright$  choose a Production
14:      print( $X \rightarrow \gamma_1 \gamma_2 \dots \gamma_n$ )
15:       $stack.pop()$ 
16:       $stack.push(\gamma_1 \gamma_2 \dots \gamma_n)$ 
17:       $X = stack.top()$ 
18:     end if
19:   end while
20: end procedure

```

题目 3(LR语法分析)

- 掌握 LR(0) 自动机的构造方法以及 SLR(1) 表格的构造方法 (参考PPT上自动机)
- 考试时会给出 LR(0) 自动机, 但需要根据分析把句柄与状态填充进去
- 掌握 LR(0) 自动机与栈之间的交互运行过程
- 不考察 LR(1)、LALR(1) 内容 (但你要掌握基本原理; 听说前几年考过, 太“变态”了)
- 参考课程录屏 16-parser-lr0-20230524、17-parser-lr1-20230531 (SLR(1) 部分)

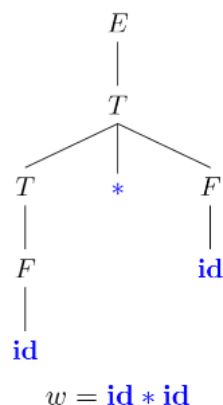
概念:

- $LL(k)$ vs. $LR(k)$
 - $LL(k)$ 在仅看到**右部前k个词法单元**时必须预测使用哪条产生式
 - $LR(k)$ 看到正在考虑的这个产生式的**整个右部**对应的词法单元之后再决定，是一个**规约**的过程
- 自底向上、不断规约、**基于句柄识别自动机的、适用于LR文法**的 LR语法分析器
 - **自底向上**: 目标: $w\$ \rightarrow S$
 - 根节点是文法的起始符号 S (约定: 第一个产生式的左边符号为起始符号)
 - 叶节点是词法单元流 $w\$$
 - 每个**中间非终结符节点**表示使用它的某条产生式进行规约

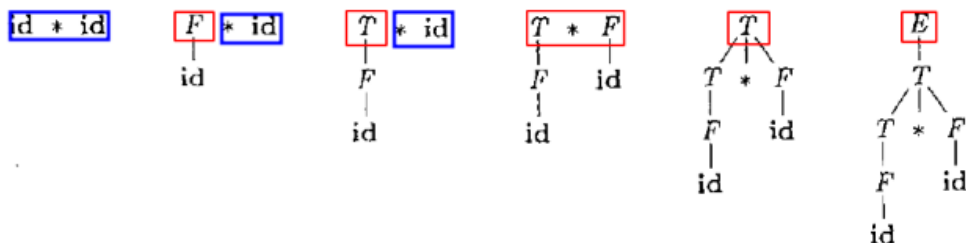
自顶向下的“推导”与 自底向上的“归约”

$$E \xRightarrow{\text{rm}} T \xRightarrow{\text{rm}} T * F \xRightarrow{\text{rm}} T * \text{id} \xRightarrow{\text{rm}} F * \text{id} \xRightarrow{\text{rm}} \text{id} * \text{id}$$

- (1) $E \rightarrow E + T$
 - (2) $E \rightarrow T$
 - (3) $T \rightarrow T * F$
 - (4) $T \rightarrow F$
 - (5) $F \rightarrow (E)$
 - (6) $F \rightarrow \text{id}$



- $E \xleftarrow{\text{rm}} T \xleftarrow{\text{rm}} T * F \xleftarrow{\text{rm}} T * \text{id} \xleftarrow{\text{rm}} F * \text{id} \xleftarrow{\text{rm}} \text{id} * \text{id}$
 - 最右推导: 先展开最右侧的非终结符, why?
 - 自底向上语法分析器提供反向最右推导 (即从最左开始规约), 因为输入是从左往右的
 - LR
 - L : 从左往右 (Left-to-right) 扫描输入
 - R : 构建反向 (Reverse) 最右 (Rightmost) 推导
- 在任意时刻, 语法分析树的**上边缘**与**剩余的输入**构成当前句型
 - LR语法分析器使用**栈**存储语法分析树的上边缘, 其包含目前所知所有信息



- 栈中的操作序列:

```
push(id);
pop(id); push(F);
pop(F); push(T);
push(*); push(id);
```

```

pop(id); push(F);

pop(F); pop(*); pop(T); push(T);

pop(T); push(E);

```

- 从**栈顶**开始寻找需要的句柄（一定是栈顶往下的连续一段）（识别的文法小一些，但是效率高）
- 两大操作：**移入输入符号**（push）与**按产生式规约**（pop和push组合操作）
- 两大问题 何时规约？按哪条产生式进行规约？
 - 何时规约与栈的当前状态相关
 - LR（严格上为SLR，见下构建时阐述）分析表：在当前状态（编号）下，面对当前文法符号，该采取什么动作

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

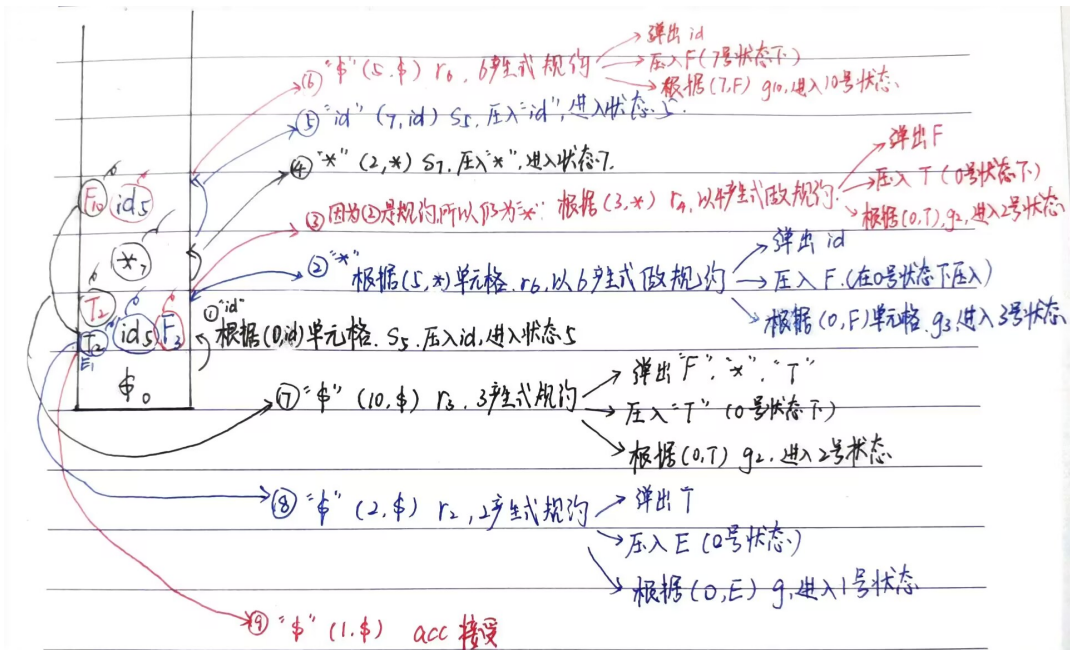
- 每行表示语法分析器当前的一个状态
- action：每列为终结符；goto：每列为非终结符
- 单元格类型：

sn	移入输入符号, 并进入状态 <i>n</i>
rk	使用 <i>k</i> 号产生式进行归约
gn	转换到状态 <i>n</i>
acc	成功接受, 结束
空白	错误

- s: shift
- r: reduce (归约)
- g: goto (右侧goto列内容均为gn)，归约的后一步骤是放入非终结符，会改变栈的状态，状态转到那里去，看右边
- acc: accept (接受)
- 空白: 出错
- 左侧为移入输入内容时的操作，右侧为规约操作时的操作
- 规约时弹出栈后当前状态变为栈顶状态（默认栈底为\$，即0号状态）

栈中每个元素的内容为当前文法符号和状态

- 一个例子: $w = id * id\$$



算法：

1. 如何构建LR(0)分析表

- LR语法分析表的内容：在当前状态（编号）下，面对当前文法符号，应该采取什么动作
- 何时规约：（必要条件）当前状态，已观察到某个产生式的完整右部
- 句柄：在输入层的（唯一）反向最右推导中，如果下一步是逆用产生式 $A \rightarrow \alpha$ 将 α 规约为 A ，则称 α 为当前句型的句柄（这里给出定义是指为了后面用句柄代替完整右部）
- LR语法分析器的关键是**高效寻找每个规约步骤使用的句柄**
- 定理：存在一种LR语法分析方法，保证句柄总是出现在栈顶（保证算法可行）

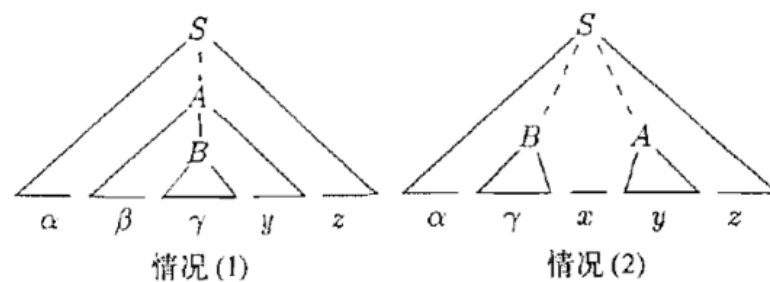
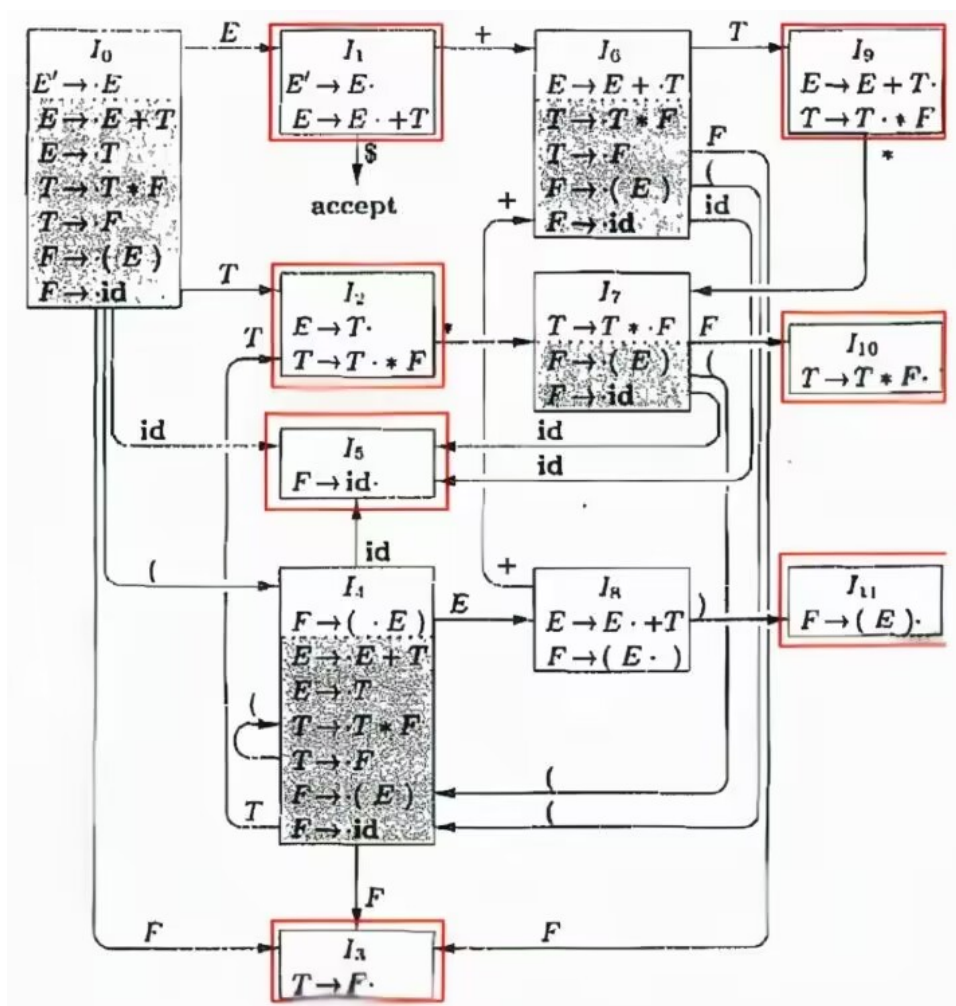


图 4-29 一个最右推导中两个连续步骤的两种情况

$$S \xrightarrow[\text{rm}]{*} \alpha Az \xrightarrow[\text{rm}]{*} \alpha \beta B y z \xrightarrow[\text{rm}]{*} \alpha \beta \gamma y z \quad S \xrightarrow[\text{rm}]{*} \alpha B x A z \xrightarrow[\text{rm}]{*} \alpha B x y z \xrightarrow[\text{rm}]{*} \alpha \gamma x y z$$

- $LR(0)$ 句柄识别有穷状态自动机



- 每个框——一个状态——一个项集
- 整张图——项集族
- 状态刻画了当前观察到的**针对所有产生式的右部的前缀**
- 定义：
 1. 项：文法 G 的一个 $LR(0)$ 项是 G 的某个产生式加上一个位于体部的点
 - 项指明语法分析器以观察到某个产生式的某个前缀（追踪某个句柄到什么程度，.左边表示已ok的）

$$A \rightarrow XYZ$$

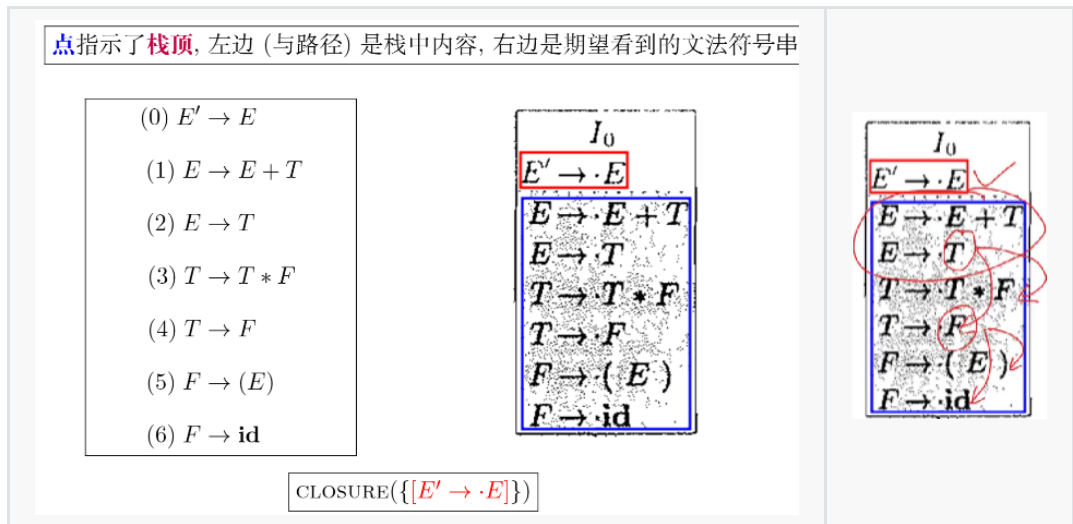
$$[A \rightarrow \cdot XYZ]$$

$$[A \rightarrow X \cdot YZ]$$

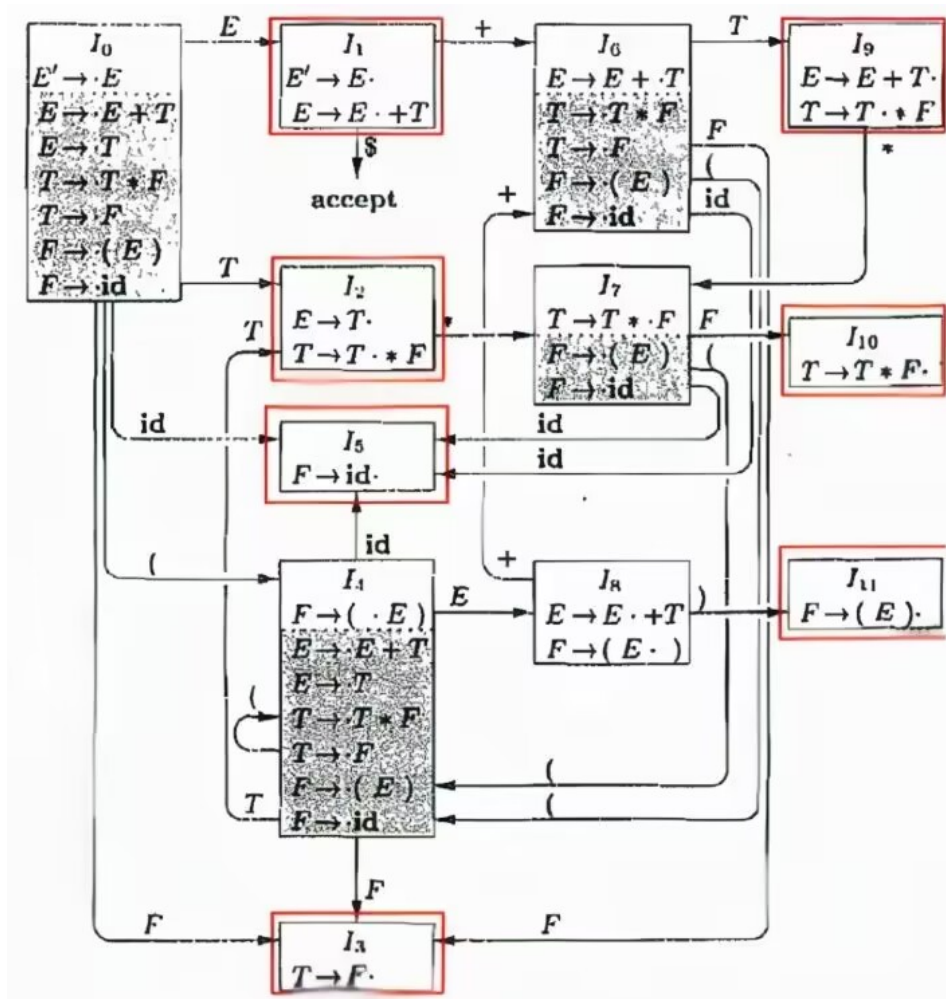
$$[A \rightarrow XY \cdot Z]$$

$$[A \rightarrow XYZ \cdot]$$
 (产生式 $A \rightarrow \epsilon$ 只有一个项 $[A \rightarrow \cdot]$)
 2. 项集：若干 项 构成的集合，一个状态可以表示为一个项集
 3. 项集族：若干 项集 构成的集合
 4. 增广文法：文法 G 的增广文法 G' 是在 G 中加入产生式 $S' \rightarrow S$ 得到的文法（仅是技术上的处理）
 - 目的是告诉语法分析器何时停止分析并接受输入符号串
 - 语法分析器当前栈中仅有 S 且面对 $\$$ ，要使用 $S' \rightarrow S$ 进行规约时，输入符号串被接受
- 步骤：

1. 初始状态：首先增广的 (0) 语句，加 .，后面几个是对非终结符不断展开得到的（闭包



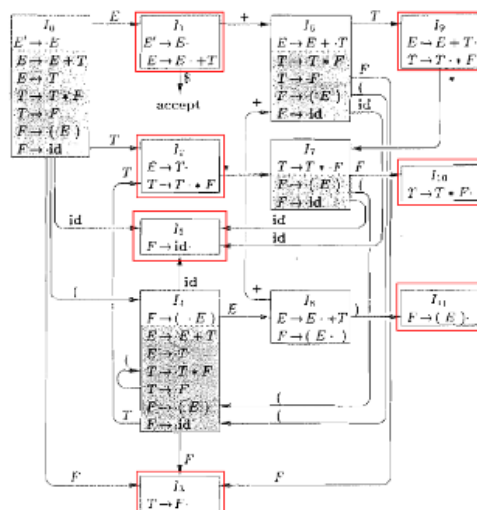
2. 状态转移：将点向后移动，将每种合法输入转化为一种状态



■ 如果 . 的后面是一个非终结符，则需要把它展开

3. 接受状态：在状态里找，如果有一个项的 . 在产生式最右端，则可能为一个句柄

4. 根据自动机构建表



	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			g1	g2	g3
1		s6				acc			
2	r2	r2	s7, r2	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			g8	g2	g3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4			g9	g3	
7	s5			s4				g10	
8		s6			s11				
9	r1	r1	s7, r1	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

- 为什么会碰到一个非终结符？（可能是在中间某一过程归约得到的，结合上面栈结构一起理解）

- 所有状态都写下来，先写s（终结符）和g（非终结符）

$$GOTO(I_i, a) = I_j \vee a \in T \Rightarrow ACTION[i, a] \leftarrow sj$$

$$GOTO(I_i, A) = I_j \vee A \in N \Rightarrow GOTO[i, A] \leftarrow gj$$

- LR(0) 看到能归约的就归约（除非左部是增广的 S' ，因为这个面对 $\$$ 就acc了），在该行填满 r_k

$$[k : A \rightarrow \alpha] \in I_i \vee A \neq S' \Rightarrow \forall t \in T \cup \{S\}. ACTION[i, t] = rk$$

- acc

$$[S' \rightarrow S.] \in I_i \Rightarrow ACTION[i, \$] \leftarrow acc$$

- 如果文法 G 的 LR(0) 分析表是没有冲突的，则 G 是 LR(0) 文法（上图就不是一个 LR(0) 文法）

- 0的含义：归约时不需向前看（整行都做归约）

LR(0) 自动机与栈之间的互动关系

向前走 $\Leftrightarrow$ 移入

回溯 $\Leftrightarrow$ 归约

自动机才是本质，栈是实现方式

- （用栈记住“来时的路”，以便回溯）

2. SLR(1) 分析表的构建

- S: simple
- 如果读到的内容不属于 $Follow(E)$ ，则即使满足规约为 E 的句柄也不可规约为 E
- 更改 LR(1) 分析表构建的第三条为

$$[k : A \rightarrow \alpha.] \in I_i \vee A \neq S' \Rightarrow \forall t \in FOLLOW(A). ACTION[i, t] = rk$$

- 定义：如果文法 G 的 SLR (1) 分析表是无冲突的，则 G 是 SLR (1) 文法

- 无冲突：ACTION表中每个单元格最多只有一种动作
- 可能冲突内容：移入/规约，规约/规约

例题：

给定下述文法 G ,

$$L \rightarrow LP \quad (1)$$

$$L \rightarrow P \quad (2)$$

$$P \rightarrow (P) \quad (3)$$

$$P \rightarrow () \quad (4)$$

(1) 简述 G 所对应的语言；

(2) 为 G 构造 $LR(0)$ 自动机；

注意：为了尽量统一状态编号，便于批改，当计算 CLOSURE 时，请按照文法编号大小顺序加入新项。当计算 $GOTO(I, X)$ 时，请按照 I 中项的出现顺序依次考虑可能的转移符号 X 。

要求：给出初始状态 I_0 的计算方法以及 $GOTO(I_0, ()$ 的计算方法。

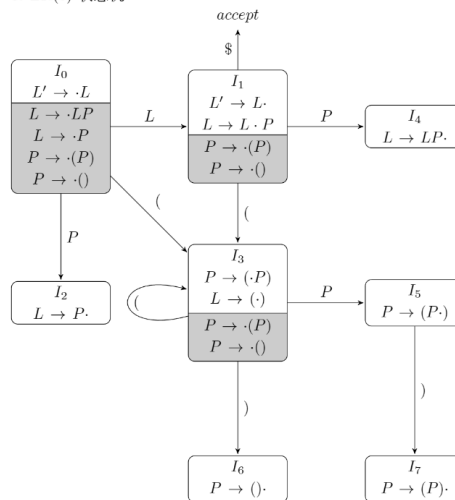
(3) 为该文法设计 $LR(0)$ 分析表；该文法是 $LR(0)$ 文法吗？请说明理由。

(4) 为该文法设计 $SLR(1)$ 分析表；该文法是 $SLR(1)$ 文法吗？请说明理由。

要求：请说明归约的设置条件。

(5) 如果该文法是 $SLR(1)$ 文法，请给出识别输入串 $(())()$ 时自动机所经历的状态（编号）。

3. $LR(0)$ 状态机



(3)

	ACTION			GOTO	
	$($	$)$	$\$$	L	P
0	s3			g1	g2
1	s3		accept		g4
2	r2	r2	r2		
3	s3	s6			g5
4	r1	r1	r1		
5		s7			
6	r4	r4	r4		
7	r3	r3	r3		

该文法是 $LR(0)$ 文法，因为 $LR(0)$ 分析表没有冲突

	ACTION			GOTO	
	$($	$)$	$\$$	L	P
0	s3			g1	g2
1	s3		accept		g4
2	r2	r2	r2		
3	s3	s6			g5
4	r1		r1		
5		s7			
6	r4	r4	r4		
7	r3	r3	r3		

$FOLLOW(L) = \{ (, \$ \}$, $FOLLOW(P) = \{ (,), \$ \}$

说明：按 $X \rightarrow \gamma$ 规约时，当前分析的非终结符号 a 要满足： $a \in FOLLOW(X)$

该文法是 $SLR(1)$ 文法，因为 $SLR(1)$ 分析表没有冲突

(4)

初始状态：0

PATH: 0 $\rightarrow$ 3 $\rightarrow$ 3 $\rightarrow$ 6 $\rightarrow$ 5 $\rightarrow$ 7 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ 3 $\rightarrow$ 6 $\rightarrow$ 4 $\rightarrow$ 1 $\rightarrow$ accept

用栈表示：0 $\xrightarrow{\text{移入}}$ 03 $\xrightarrow{\text{移入}}$ 033 $\xrightarrow{\text{移入}}$ 0336 $\xrightarrow{\text{规约}}$ 035 $\xrightarrow{\text{移入}}$ 0357 $\xrightarrow{\text{规约}}$ 02 $\xrightarrow{\text{规约}}$ 01 $\xrightarrow{\text{移入}}$ 013 $\xrightarrow{\text{移入}}$ 0136 $\xrightarrow{\text{规约}}$ 014 $\xrightarrow{\text{规约}}$ 01 $\rightarrow$ accept

题目 4(ANTLR4与“优先级上升算法”)

- 能够根据优先级上升算法改造给定文法
- 能够给出给定输入在改造后的文法下对应的语法分析树
- 不考查 ANTLR 4 AllStar 算法的其余内容
- 参考课程录屏 7-parser-ll-8-parser-allstar-20230407

算法：

- 从例子入手：

```
stat : expr ';' EOF;

expr : expr '*' expr
      | expr '+' expr
      | INT
      | ID
      ;
```

```
E(int _p){
  match INT|ID
  while(true){
    if(4>=_p && match *){
      E(5)
    }else if (3>=_p && match +){
      E(4)
    }else
      return;
  }
}
```

```
expr[ int _p ]
: ( INT
  | ID
  )
  ( {4 >= $_p}? '*' expr[5]
  | {3 >= $_p}? '+' expr[4]
  )*
;
```

○ 关注几个点:

- 蓝色 (没有左递归的部分) 没有特殊处理
- 红色 (有左递归的部分), 注意最后的 *, 代表循环, 本质上是把一个左递归的文法改造为循环迭代, 即 $(INT|ID) ((+|*)(INT|ID))^*$

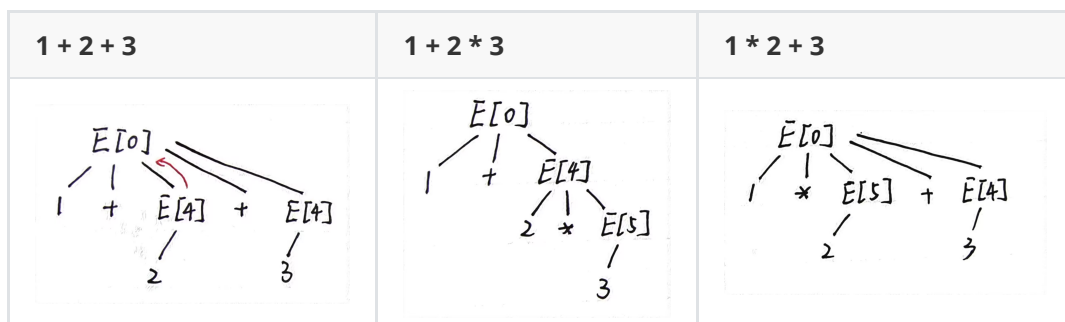
```
1 E(_p){
2   match ID|INT;
3   while {
4     if(4 >= _p) { match '*'; E(5);}
5     if(3 >= _p) { match '+'; E(4);}
6   }
7 }
```

○ 引入了一个参数——指优先级

- 如果是外部调用的, 则参数为0
- 4,3,5是怎么来的? 以及为什么需要 $\{4 \geq _p\}$? 的判断? 为什么后面的表达式变高了?
- 优先级的产生根据给定的文法, 从下往上优先级越来越高


```
expr : expr '*' expr 4
      | expr '+' expr 3
      | INT           2
      | ID            1
      ;
```
- 每match一次, 就是在往下走一层, 越往下就意味着优先级越高, 所以我要展开加法, 就需要此处的加法优先级高一些, 这就是为什么需要进行判断的原因
- 进入了加法后, 如果一符号能在加法中展开, 则意味着这个符号的优先级比加法高, 所以需要把优先级向上升一级

○ 在此改造后文法上的一些例子



● 根本问题:

- 究竟是在expr的单签调用中匹配下一个运算符
- 还是让wxpr的调用者匹配下一个运算符

- 更多例子来探索参数的设置——即什么时候需要上升？

<pre>stat : expr ';' EOF; expr : expr '*' expr expr '+' expr '(' expr ')' INT ID ;</pre>	<pre>stat : expr ';' EOF; expr : '-' expr '!' expr expr '+' expr ID ;</pre>	<pre>stat : expr ';' EOF ; expr : <assoc = right> expr '^' expr expr '+' expr INT ;</pre>
<pre>expr[int _p] : ('(' expr[0] ')') INT ID) ({5 >= \$_p}? '*' expr[6] {4 >= \$_p}? '+' expr[5])*</pre>	<pre>expr[int _p] : (ID '-' expr[4]) ({3 >= \$_p}? '!' {2 >= \$_p}? '+' expr[3])*</pre>	<pre>expr[int _p] : (INT) ({3 >= \$_p}? '^' expr[3] {2 >= \$_p}? '+' expr[3])*</pre>
这个文法不是左递归的，因为 (是终结符，所以框中语句和 INT ID 是一样的为什么参数为0？ 因为这里调用expr和外面没什么区别	- 是前缀运算符，是右结合的 ! ! 是后缀运算符，是左结合的，并且它是一元运算符	二元运算符，并且右结合

- 对于左结合，调用一次后，后面的expr参数就要上升，使之不能在内部展开
- 对于右结合，参数不变化（见第三列的 ^，和第二列的 -）
- 注意不能递归继续展开向上回溯的时候，要返回当初进入时的位置

- 注意三个部分：终结符、非终结符、*
- 例子

考虑如下表达式文法，其中 ! 为一元后缀运算符，^ 为右结合运算符。

$$S \rightarrow E$$

$$E \rightarrow E! \mid E \wedge E \mid E + E \mid \text{INT}$$

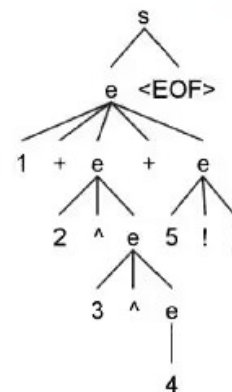
(1) 请给出 ANTLR4 使用优先级上升算法改造后的文法。

(2) 请给出 $1 + 2 \wedge 3 \wedge 4 + 5!!$ 在上述文法下对应的语法分析树。请给出关键的解释，点到即可，不必面面俱到。

(1) 改造后的文法如下图所示：

```

8  s : e (0) EOF;
9
10 e[int _p]
11   : ( INT )
12     ( {3 >= $_p}? '^' e[3]
13       | {2 >= $_p}? '+' e[3]
14       | {4 >= $_p}? '!'
15       ) (0)
16   ;
```



题目 5(语法制导的翻译)

- 语法制导的翻译
- 不考查 ANTLR 4 中的写法
- 参考课程录屏 10-semantics-ag-20230421、10-semantics-ag-20230426

概念:

- 属性文法: 为上下文无关文法赋予语义
- offline方式计算属性值: 已有语法分析树, 按照从左到右的深度优先顺序遍历语法分析树
 - 关键: 在合适的时机执行合适的动作, 计算相应的属性值
- online方式计算属性值: 在构建语法分析树的时候把属性顺带算出来, 语义动作嵌入的位置决定了何时执行该操作

- 基本思想: 一个动作在它左边的所有文法符号都处理过之后立即执行

```
22 prog : stat+ ;
23
24 stat : expr      { System.out.println($expr.val); }
25      | ID '=' expr { memory.put($ID.text, $expr.val); }
26      ;
27
28 expr returns [int val]
29      : l = expr op = ('*' | '/') r = expr { $val = eval($l.val, $r.val, $op.type); }
30      | l = expr op = ('+' | '-') r = expr { $val = eval($l.val, $r.val, $op.type); }
31      | '(' expr ')' { $val = $expr.val; }
32      | ID           { $val = memory.getOrDefault($ID.text, 0); }
33      | INT          { $val = $INT.int; }
34      ;
```

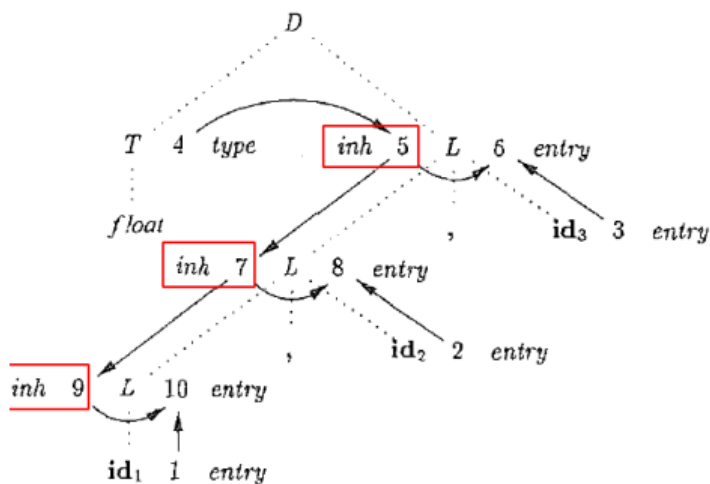
- 综合属性: 节点N上的综合属性只能通过N的子节点或N本身属性来定义
 - 依赖关系自底向上, 把动作放在产生式的最后 (如上)
- S属性定义: 如果一个SDD的每个属性都是综合属性, 则它是S属性定义
 - 此类属性值的计算可以在自顶向下的 LL 语法分析过程中实现
 - 在 LL 语法分析器中, 递归下降函数 A 返回时, 计算相应节点 A 的综合属性值
- 继承属性: 只能通过N的父节点、N本身和N的兄弟节点上的属性来定义

产生式	语义规则
1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$
5) $L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$

float id₁, id₂, id₃

- T、L是分开, d₁, d₂, d₃怎么知道 float 类型?

把 float 作为 L 递归调用的参数传进去 (继承属性: $L.inh = T.type$)



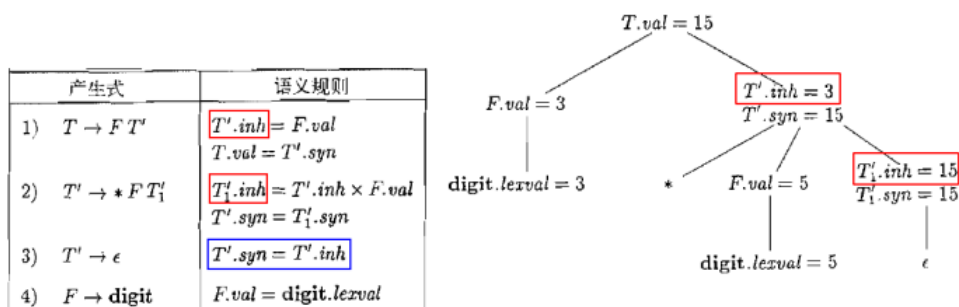
```

7 decl : type vars[$type.text] ;
8 type : 'int'      # IntType
9       | 'float'   # FloatType
10      ;
11 vars [String typeStr]
12      : ID { System.out.println($ID.text + " : " + $typeStr); }
13      (',' ID { System.out.println($ID.text + " : " + $typeStr); })*
14      ;

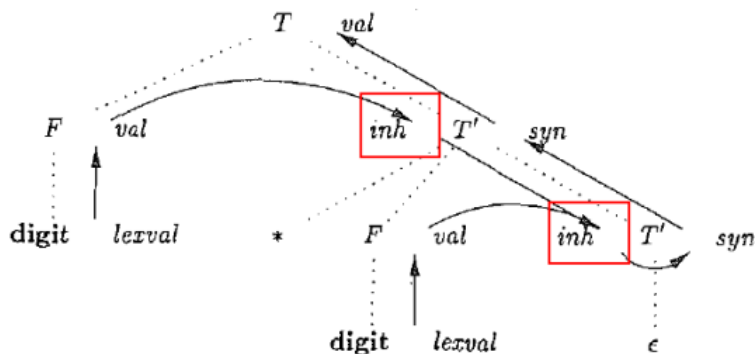
```

- 例子——连乘，需要使用继承属性（两个乘数不在一起）

继承属性 $T'.inh$ 用于在表达式中从左向右传递中间计算结果



3 * 5



- 信息流向：先从左往右、从上往下传递信息，在从下往上传递信息

- L属性定义

如果一个 SDD 的每个属性

- (1) 要么是综合属性,
- (2) 要么是继承属性, 但是它的规则满足如下限制:
对于产生式 $A \rightarrow X_1 X_2 \dots X_n$ 及其对应规则定义的继承属性 $X_i.a$, 则这个规则只能使用
 - (a) 和产生式头 A 关联的继承属性;
 - (b) 位于 X_i 左边的文法符号实例 $X_1, X_2, \dots, X_{i-1}$ 相关的继承属性或综合属性;
 - (c) 和这个 X_i 的实例本身相关的继承属性或综合属性, 但是在由这个 X_i 的全部属性组成的依赖图中不存在环。

则它是 L 属性定义。

◦ 为什么加限制?

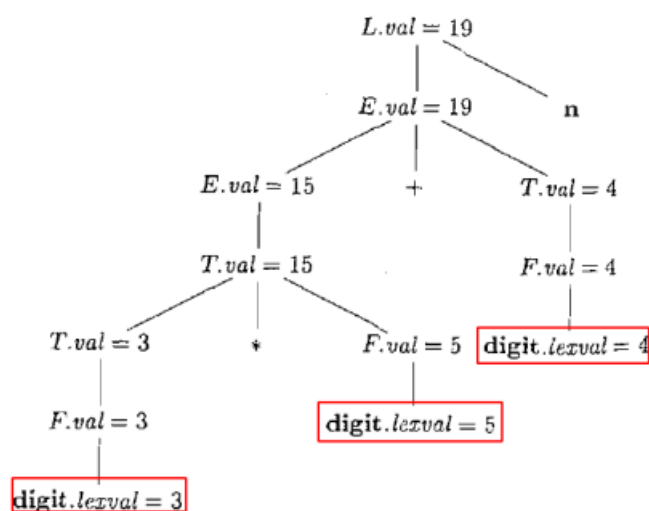
- 如果不加“a.只能依赖父节点的继承属性”, 因为父节点的综合属性依赖子节点, 不加限制很容易成环
 - “b.只能依赖左兄弟节点的属性 (综合、继承)”, 因为右侧节点还没有构造好, 但左侧都已计算好了
 - 节点的属性可以依赖本身其他属性, 但要注意不能存在环
- 语法制导定义SDD——是一个上下文无关文法和属性及集合的规则

每个文法符号都可以关联多个属性

产生式	语义规则
1) $L \rightarrow E n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

每个产生式都可以关联一组规则

- SDD唯一确定了语法分析树上每个非终结符节点的属性值
 - SDD没有规定以什么方式、什么顺序计算这些属性值
- 注释语法分析树: 显示了各个属性值的语法分析树



$3 * 5 + 4$

- SDT语法制导的翻译方案——其产生体中嵌入语义动作的上下文方案

产生式	语义规则
1) $L \rightarrow E n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

$L \rightarrow E n$	$\{ \text{print}(E.val); \}$
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$
$E \rightarrow T$	$\{ E.val = T.val; \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$
$T \rightarrow F$	$\{ T.val = F.val; \}$
$F \rightarrow (E)$	$\{ F.val = E.val; \}$
$F \rightarrow \text{digit}$	$\{ F.val = \text{digit.lexval}; \}$

o 如何把一个SDD转换为SDT?

1. 对于S属性定义

S 属性定义

产生式	语义规则
1) $L \rightarrow E n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

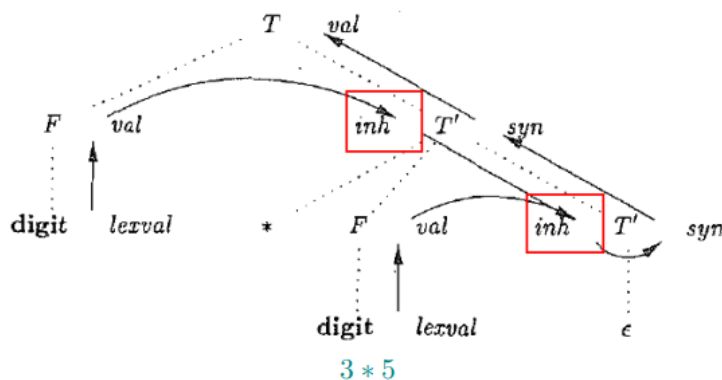
后缀翻译方案

$L \rightarrow E n$	$\{ \text{print}(E.val); \}$
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$
$E \rightarrow T$	$\{ E.val = T.val; \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$
$T \rightarrow F$	$\{ T.val = F.val; \}$
$F \rightarrow (E)$	$\{ F.val = E.val; \}$
$F \rightarrow \text{digit}$	$\{ F.val = \text{digit.lexval}; \}$

后缀翻译方案: 所有动作都在产生式的最后

2. 对于L属性定义

L 属性定义 与 LL 语法分析



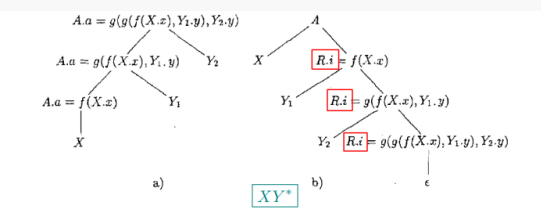
$$A \rightarrow X_1 \cdots X_i \cdots X_n$$

原则: 从左到右处理各个 X_i 符号

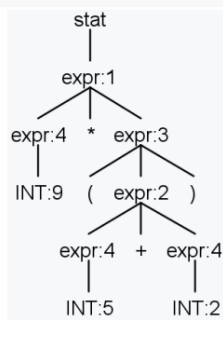
对每个 X_i , 先计算继承属性, 后计算综合属性

算法:

- 感觉这部分比较灵活, 也没有很固定的套路, 结合着几个例子看一下思路吧
- 另外, 例题中都是使用ANTLR4语法直接实现了, 注意思考怎么用语义规则书写
- 递归下降子过程 $A \rightarrow X_1 \cdots X_i \cdots X_n$
 - 在调用 X_i 子过程之前, 计算 X_i 的继承属性, 以 X_i 的继承属性为参数调用 X_i 子过程
 - 在 X_i 子过程返回之前, 计算 X_i 的综合属性, 在 X_i 子过程结束时返回 X_i 的综合属性

 (左递归) S 属性定义 $A \rightarrow A_1 Y \quad A.a = g(A_1.a, Y.y)$ $A \rightarrow X \quad A.a = f(X.x)$ (右递归) L 属性定义 $A \rightarrow X R \quad R.i = f(X.x); A.a = R.s$ $R \rightarrow Y R_1 \quad R_1.i = g(R.i, Y.y); R.s = R_1.s$ $R \rightarrow \epsilon \quad R.s = R.i$	(右递归) L 属性定义 $A \rightarrow X R \quad R.i = f(X.x); A.a = R.s$ $R \rightarrow Y R_1 \quad R_1.i = g(R.i, Y.y); R.s = R_1.s$ $R \rightarrow \epsilon \quad R.s = R.i$ 原则: 继承属性在处理文法符号之前, 综合属性在处理文法符号之后 L 属性定义的 SDT $A \rightarrow X \quad \{R.i = f(X.x)\} \quad R \quad \{A.a = R.s\}$ $R \rightarrow Y \quad \{R_1.i = g(R.i, Y.y)\} \quad R_1 \quad \{R.s = R_1.s\}$ $R \rightarrow \epsilon \quad \{R.s = R.i\}$
$A \rightarrow X \quad \{R.i = f(X.x)\} \quad R \quad \{A.a = R.s\}$ $R \rightarrow Y \quad \{R_1.i = g(R.i, Y.y)\} \quad R_1 \quad \{R.s = R_1.s\}$ $R \rightarrow \epsilon \quad \{R.s = R.i\}$ <pre> 1: procedure A() 2: if token = ? then 3: X.x ← MATCH(X) 4: R.i ← f(X.x) 5: R.s ← R(R.i) 6: return R.s </pre> ▷ A 是开始符号, 无需继承属性做参数 ▷ 假设选择 $A \rightarrow X R$ 产生式 ▷ 假设 X 是终结符, 返回综合属性 ▷ 先计算 $R.i$ 继承属性 ▷ 递归调用子过程 $R(R.i)$ ▷ 返回 $A.a \leftarrow R.s$ 综合属性	$A \rightarrow X \quad \{R.i = f(X.x)\} \quad R \quad \{A.a = R.s\}$ $R \rightarrow Y \quad \{R_1.i = g(R.i, Y.y)\} \quad R_1 \quad \{R.s = R_1.s\}$ $R \rightarrow \epsilon \quad \{R.s = R.i\}$ <pre> 1: procedure R(R.i) 2: if token = ? then 3: Y.y ← MATCH(Y) 4: R_1.i ← g(R.i, Y.y) 5: R.s ← R(R_1.i) 6: return R.s 7: else if token = ? then 8: return R.i </pre> ▷ R 使用继承属性 $R.i$ 做参数 ▷ 假设选择 $R \rightarrow Y R$ 产生式 ▷ 假设 Y 是终结符, 返回综合属性 ▷ 先计算 $R.i$ 继承属性 ▷ 递归调用子过程 $R(R.i)$ ▷ 返回综合属性 ▷ 假设选择 $R \rightarrow \epsilon$ 产生式 ▷ 返回 $R.s \leftarrow R.i$ 综合属性

1. 后缀表达

Definition (后缀表示 (Postfix Notation)) (1) 如果 E 是一个 变量或常量, 则 E 的后缀表示是 E 本身; (2) 如果 E 是形如 $E_1 \text{ op } E_2$ 的表达式, 则 E 的后缀表示是 $E_1' E_2' \text{ op}$, 这里 E_1' 和 E_2' 分别是 E_1 与 E_2 的后缀表达式; (3) 如果 E 是形如 (E_1) 的表达式, 则 E 的后缀表示是 E_1 的后缀表示。 $(9 * 5) + 2 \Rightarrow 95 * 2 +$ $9 * (5 + 2) \Rightarrow 952 + *$	 $9 * (5 + 2) \Rightarrow 952 + *$
--	--

S属性文法, 只有一综合属性

```

7 stat : expr { System.out.println($expr.postfix); }
8
9
10 expr returns [String postfix]
11   : l = expr op = '*' r = expr { $postfix = $l.postfix + $r.postfix + $op.text; }
12   | l = expr op = '+' r = expr { $postfix = $l.postfix + $r.postfix + $op.text; }
13   | '(' expr ')' { $postfix = $expr.postfix; }
14   | INT { $postfix = $INT.text; }
15   ;

```

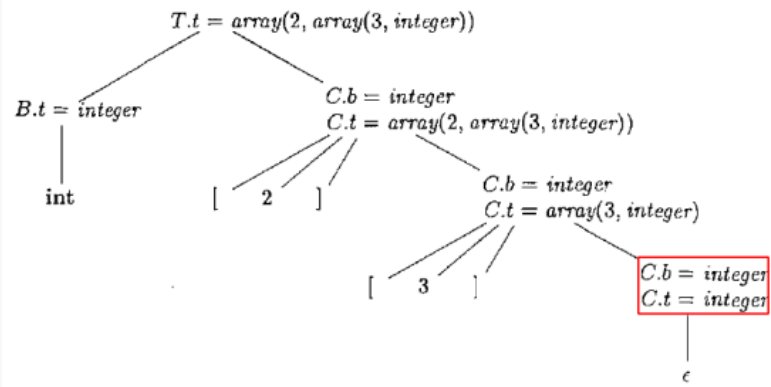
2. 数组类型

产生式
$T \rightarrow B C$
$B \rightarrow \text{int}$
$B \rightarrow \text{float}$
$C \rightarrow [\text{num}] C_1$
$C \rightarrow \epsilon$

`int[2][3]`

类型表达式 (2, (3, int))

继承属性 $C.b$ 将一个基本类型沿着树向下传播



`int[2][3]`

综合属性 $C.t$ 收集最终得到的类型表达式

```

7 // OR: type ID ('[' INT '])* ';'
8 arrDecl : basicType ID arrayType[$basicType.text]
9 { System.out.println($ID.text + " : " + $arrayType.array_type); } ';' ;
10
11 arrayType[String basic_type]
12 returns [String array_type]
13 : '[' INT ']' arrayType[$basic_type]
14 { $array_type = "(" + $INT.int + ", " + $arrayType.array_type + ")"; }
15 | { $array_type = $basic_type; }
16 ;

```

ArrayTypeAG.g4

3. 数组声明、数组引用、类型检查

```

14 stat : varDecl
15     { String id = $varDecl.ctx.ID().getText();
16       System.out.println(id + " : " + typeMap.get(id)); }
17 | arrDecl
18     { String id = $arrDecl.ctx.ID().getText();
19       System.out.println(id + " : " + typeMap.get(id)); }
20 | lhs = expr '=' rhs = expr ';'
21     { System.out.println($lhs.type + " <=> " + $rhs.type); }
22 ;

24 varDecl : basicType ID ';'
25     { typeMap.put($ID.text, $basicType.text); };
26 basicType : 'int' | 'float' ;
27
28 // OR: type ID ('[' INT '])* ';'
29 arrDecl : basicType ID arrayType[$basicType.text]
30     { typeMap.put($ID.text, $arrayType.array_type); }
31     ';' ;
32 arrayType[String basic_type]
33     returns [String array_type]
34     : '[' INT ']' arrayType[$basic_type]
35     { $array_type = "(" + $INT.int + ", " + $arrayType.array_type + ")"; }
36 | { $array_type = $basic_type; }
37 ;

39 expr returns [String type]
40     : ID { String expr_type = typeMap.get($ID.text); }
41     ('[' INT ']')
42     {
43         int start = expr_type.indexOf(',');
44         int end = expr_type.lastIndexOf(')');
45         expr_type = expr_type.substring(start + 1, end);
46     }
47     )* { $type = expr_type; }
48 | ID { $type = typeMap.get($ID.text); }
49 | INT { $type = "int"; }
50 ;

```

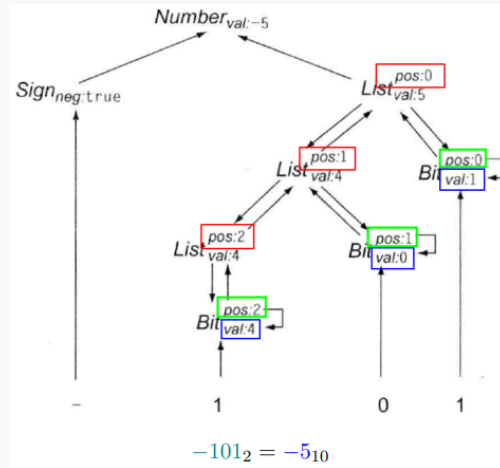
4. 二进制

$P =$	$Number$	$\rightarrow$	$Sign\ List$	$T = \{+, -, 0, 1\}$
	$Sign$	$\rightarrow$	$+$	
		$ $	$-$	$NT = \{Number, Sign, List, Bit\}$
	$List$	$\rightarrow$	$List\ Bit$	
		$ $	Bit	$S = \{Number\}$
	Bit	$\rightarrow$	0	
		$ $	1	

$$-101_2 = -5_{10}$$

有符号二进制数 L 属性定义

	产生式	属性规则
1	$Number \rightarrow Sign\ List$	$List.position \leftarrow 0$ if $Sign.negative$ then $Number.value \leftarrow -List.value$ else $Number.value \leftarrow List.value$
2	$Sign \rightarrow +$	$Sign.negative \leftarrow false$
3	$Sign \rightarrow -$	$Sign.negative \leftarrow true$
4	$List \rightarrow Bit$	$Bit.position \leftarrow List.position$ $List.value \leftarrow Bit.value$
5	$List_0 \rightarrow List_1\ Bit$	$List_1.position \leftarrow List_0.position + 1$ $Bit.position \leftarrow List_0.position$ $List_0.value \leftarrow List_1.value + Bit.value$
6	$Bit \rightarrow 0$	$Bit.value \leftarrow 0$
7	$Bit \rightarrow 1$	$Bit.value \leftarrow 2^{Bit.position}$



5. 作业题（重点看）

考虑如下文法 G 。

```

P → D
D → D ; D
D → T id
D → func id {D}
T → int

```

请设计语法制导的翻译方案，完成下列任务。你需要自行定义合适的属性。

- 打印程序 P 一共声明了多少个 id （包括 $T\ id$ 与 $func\ id$ ）。
例如，句子 $int\ id; func\ id\{int\ id\}$ 中共声明了 3 个 id 。
- 打印程序 P 中每个变量 id （不考虑函数名 id ）在 $\{\}$ 中的嵌套深度。
例如，在句子 $int\ id; func\ id\{int\ id\}$ 中，第一个变量 id 的嵌套深度为 0，第二个变量 id 的嵌套深度为 1。

解答：

- (1) $count$ 表示 id 的个数

$P \rightarrow D \{print(D.count)\}$
$D \rightarrow D_1 ; D_2 \{D.count = D_1.count + D_2.count\}$
$D \rightarrow T\ id \{D.count = 1\}$
$D \rightarrow func\ id \{D_1\} \{D.count = D_1.count + 1\}$
$T \rightarrow int \{ \}$

- (2) $depth$ 表示 id 的深度

$P \rightarrow \{D.depth = 0\} D$
$D \rightarrow \{D_1.depth = D.depth; D_2.depth = D.depth\} D_1 ; D_2$
$D \rightarrow T\ id \{print(D.depth)\}$
$D \rightarrow \{D_1.depth = D.depth + 1\} func\ id \{D_1\}$
$T \rightarrow int \{ \}$

考虑如下文法 G ，它定义了一种二叉树结构，其中每个内部节点（非 nil 节点）包含一个数字与左右两个子树：

```

T → nil | (T int T)

```

其中， int 是词法单元。

请设计语法制导的翻译方案，完成下列任务。你需要自行定义合适的属性。

- 判断一个二叉树是否是平衡的。
(某二叉树是平衡的，当且仅当每个内部节点的左右子节点的高度差不大于 1。假设 nil 节点的高度为 0。)
- 判断一个二叉树是否是二叉搜索树。
(某二叉树是二叉搜索树，当且仅当对于每个内部节点 N ，它的左子树中所有内部节点中的数字都小于 N 中的数字，而它的右子树中所有内部节点中的数字都不小于 N 中的数字。)

- (1)

```

1) T → nil {T.height = 0; T.balance = true;}
2) T → (T1 int T2) {T.height = max(T1.height, T2.height) + 1;
    T.balance = T1.balance && T2.balance && |T1.height - T2.height| ≤ 1;}

```

- (2)

```

(1) T → nil {T.search = ture; T.max_val = T.min_val = null;}
(2) T → (T1 int T2) {if(T1.max_val == null && T2.max_val == null){
    T.search = ture; T.max_val = T.min_val = int.val;}
    else if(T1.max_val == null){
        continued on next page
    }
}

```

continued from previous page

```

T.search = T2.search && (T2.min_val ≥ int.val); T.min_val = int.val;
T.max_val = T2.max_val;
else if(T2.max_val == null){
    T.search = T1.search && (T1.max_val < int.val); T.min_val = T1.min_val;
    T.max_val = int.val;
}
else{
    T.search = T1.search && T2.search &&
    (T1.max_val < int.val && T2.min_val ≥ int.val);
    T.min_val = T1.min_val; T.max_val = T2.max_val;
}

```

6. more about 二进制

以下文法 G 描述了带可选小数部分的二进制数。我们希望通过属性文法计算二进制数对应的十进制表示。

$$\begin{aligned} N &\rightarrow L.L \\ N &\rightarrow L \\ L &\rightarrow LB \\ L &\rightarrow B \\ B &\rightarrow 1 \\ B &\rightarrow 0 \end{aligned}$$

- (1) 请给出一种 S 属性翻译方案。
提示: $123 = (1 \times 10 + 2) \times 10 + 3$
- (2) 请给出一种 L 属性翻译方案。

(1) w 表示二进制串宽度

$N \rightarrow L_1.L_2\{N.val = L_1.val + 2^{-L_2.w} * L_2.val\}$
$N \rightarrow L\{N.val = L.val\}$
$L \rightarrow L_1B\{L.w = L_1.w + 1, L.val = L_1.val * 2 + B.val\}$
$L \rightarrow B\{L.w = 1, L.val = B.val\}$
$B \rightarrow 1\{B.val = 1\}$
$B \rightarrow 0\{B.val = 0\}$

(2) $L.val$ 的值由 $B.val$ 获得, $L.int = true$ 表示 L 是整数部分, $L.w$ 表示 L 的二进制数宽度, $B.inhVal$ 表示 B 左边的串的二进制整数值, $B.inhW$ 表示 B 左边的二进制位数

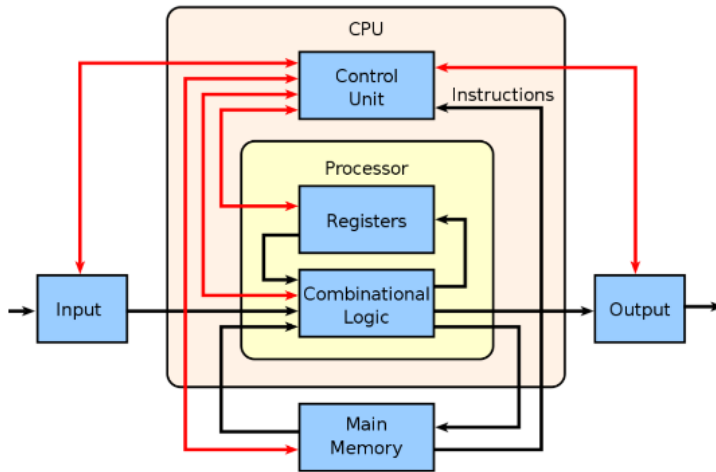
$N \rightarrow \{L_1.int = true\}L_1.\{L_2.int = false\}L_2\{N.val = L_1.val + L_2.val\}$
$N \rightarrow \{L.int = true\}L\{N.val = L.val\}$
$L \rightarrow \{L_1.int = L.int\}L_1\{B.inhVal = L_1.val, B.inhW = L_1.w, B.inhInt = L.int\}$ $B\{L.w = L_1.w + 1, L.val = B.val\}$
$L \rightarrow \{B.inhVal = 0, B.inhW = 0, B.inhInt = L.int\}B\{L.w = 1, L.val = B.val\}$
$B \rightarrow 1 \{ \text{if } B.inhInt == true$ $\text{then } B.val = B.inhVal * 2 + 1$ $\text{else } B.val = B.inhVal + 2^{-B.inhW-1}$
$B \rightarrow 0 \{ \text{if } B.inhInt == true$ $\text{then } B.val = B.inhVal * 2$ $\text{else } B.val = B.inhVal\}$

题目 6(目标代码生成)

- 给定一段 C 语言程序 (含过程调用) 与对应的 RISC-V 代码片段, 填充缺失的代码行 (提供英文注释)。
- 熟悉 RISC-V 基本指令, 需要对整个代码理解 (会给定指令限用范围), 尤其关于栈、返回值等
- 参考课程录屏 18-codegen-riscv-20230602、19-codegen-riscv-20230607、助教的 PPT

概念:

- RISC-V: 第五代精简指令集计算机, 精简在每个指令只做一件事
- 放一张计算机组织



寄存器

寄存器名	用途
zero	存储常数0，不允许修改
ra	return address，存储返回地址，函数调用之前存放下一条指令地址
t ₀ ~t ₆	临时变量保存
pc	程序计数器，保存下一条要执行指令的地址
a ₀ ~a ₇	用与存放函数调用的参数

- 所有运算都发生在寄存器中，内存中的值要先load到寄存器中

指令

1. 伪指令

指令	含义	示例
li	load immediate，加载立即数到寄存器 (实现：addi dest, dest, immediate)	li t0, 20 #加载20到t0
mv	(实现：add dest, src, zero)	mv a0, t6 #加载t6的值到a0

- mv 实际实现由add dest, src, zero实现

2. 基本指令

指令	含义	示例
addi	add immediate，加立即数 subi不存在，可以使用addi一个负数实现	addi t0, t0, 20
add	三个参数，分别是目标寄存器，和两个操作数所在的寄存器 存在sub指令	add t2, t0, t1 #t0的值+t1的值存入t2
la	load address，加载地址到寄存器	la t0, g
lw	load word，加载word到寄存器	lw t0, 0(t0) 0: 偏移量 (t0): 基址
sw	store word，把寄存器的值存入地址对应空间	sw t6, 0(t0)

系统调用

How to use SYSCALL system services

Step 1. Load the service number in register a7.

I

Step 2. Load argument values, if any, in a0, a1, a2, a3, fa0, ... as specified.

Step 3. Issue the ECALL instruction.

Step 4. Retrieve return values, if any, from result registers as specified. Example: display the

```
li a7, 1 # service 1 is print integer
add a0, t0, zero # load desired value into argument register a0, using pseudo-op
ecall
```

1. 把 service number 放到 a₇ 寄存器中

2. 把系统调用的参数放到 $a_0, a_1, a_2, a_3, \dots a_7$

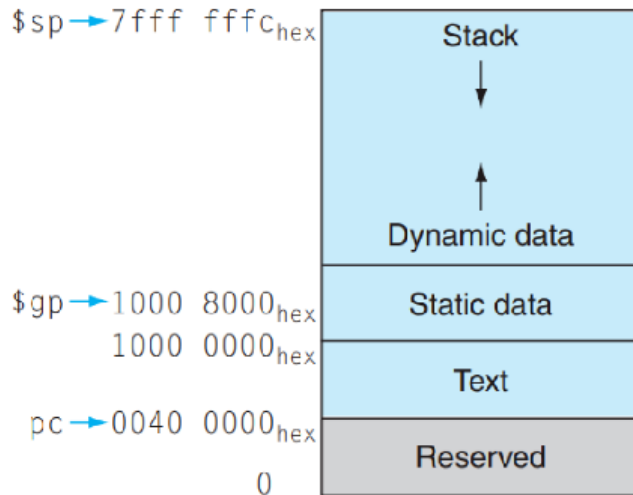
`add a0, t0, zero`: a0的值等于t0的值+0, 相当于是在做mov, 提供了伪指令mov

3. 调用指令 `ecall`

4. 获取返回值

- 打印字符串用 4 号调用, 把msg的首地址 (用la得到) 放到a0中

代码分区



静态数据区 .data

- 加点的指令是汇编指令, 是写给编译器看的
- `g: .word 0`
 - `g`——label, 分配空间的一块地址, g是地址的标号
 - `.word`——告诉占用空间的大小
 - `.word`: 4 bytes
 - `.half`: 2bytes
 - `.dword`: 8bytes
- `msg: .string "result is:"`
 - `.string`: 最后会以 `\0` 结尾
 - `.ascii`: 没有 `\0` 结尾
- `numbers: .word 20, 10, 0, -10`: 数组

代码示例

max-if: (跳转指令)

```
1  .data
2  a: .word 100
3  b: .word 200
4  c: .word 0
5
6  .text
7  # la t0, a
8  # lw t0, 0(t0)
9  # 等价于下面这行, 简洁写法, 不是RISC标准, 不一定支持
10 lw t0, a
11 lw t1, b #同上, 可以拆分
```

```

12
13 bge t0, t1, greater_equal # 如果t0数据大于等于t1数据, 跳转到greater_equal
14 mv t2, t1
15 j end # j: jump, 伪指令
16
17 greater_equal:
18 mv t2, t0
19 # t2用于保存最大值
20
21 end:
22 # la t3, c
23 # sw t2 0(t3)
24 # 以上两行可以由下面这行代替, 同样非RISC标准
25 sw t2, c, t3

```

array-for: (for循环遍历数组)

```

1 .data
2 numbers: .word -30, 30, -20, 20, -10, 10, 0
3 size: .word 7
4
5 positive_sum: .word 0
6 negative_sum: .word 0
7
8 .text
9 la t0, numbers # t0: address of array, 拿到首地址
10 lw t1, size # t1: size = 7
11
12 mv t2, zero # counter, 下标初始为0
13
14 li t3, 0 # sum of positive numbers
15 li t4, 0 # sum of negative numbers
16
17 loop:
18 bge t2, t1, end_loop
19 # 以下三条指令实现numbers[t2]
20 # mul t5, t2, 4 和下面一句相同, 但下面效率高 t2 * 4 -> t5
21 slli t5, t2, 2 #shift left logical immediate,
22 add t5, t0, t5 # t5得到元素内存地址
23 lw t5, 0(t5)
24
25 addi t2, t2, 1 # i++
26
27 bltz t5, negative #branch if less than zero
28 add t3, t3, t5
29 j loop
30
31 negative:
32 add t4, t4, t5
33 j loop
34
35 end_loop:
36 sw, t3, positive_sum, t5 # 拆, t5是中介
37 sw, t4, negative_sum, t6 # 拆, t6是中介

```

proc-max: (函数调用)

```
1  .data
2  max_result: .word 0
3
4  .text
5  .global main # 声明为全局函数
6
7  # 约定返回值要放到a0, a1里
8  max:
9  blt a0, a1, smaller #lt less than
10 j end_max
11
12 smaller:
13 mv a0, a1 #返回值放入a0
14
15 end_max:
16 ret # 伪指令, 与下面一行等价
17 # jalr zero 0(ra)
18 # 释义: jalr: jump and link register 填zero: 丢弃max的下一条指令地址, 类比ra
19 # 等价伪指令: jr ra #jr: jump register
20
21 ##### main #####
22 .data
23 a: .word 100
24 b: .word 200
25
26 .text
27 main:
28 # 为函数准备参数, 寄存器有规定, 超过8个参数, 放到栈里
29 lw a0, a
30 lw a1, b
31
32 call max # 伪指令, 与下面一行等价
33 # jal ra, max
34 # 释义: jal: jump and link ra: return address register, 存储返回地址
35 # pc指到max的第一条指令, main后面一条指令的地址存到ra
36 # 同价伪指令: jal max
37
38 sw a0, max_result, t0
```

阶乘: (函数递归调用)

```
1  .text
2  .global main
3
4  #结果过大, a0放不下, 则将a0, a1拼起来使用
5  factorial:
6  beqz a0, base_case
7
8  # 调用者保存
9  addi sp, sp, -8 # 地址从高到低, 所以要减, 每保存一个-4
10 sw a0, 4(sp) # 保存a0
11 sw ra, 0(sp) # 保存ra, 被调用函数调用其他函数时, ra被覆写, 需要注意维护
12
13 # n > 0: 递归调用
14 addi a0, a0, -1 # a0: k - 1
```

```

15  call factorial    # a0: factorial(k - 1)
16  mv t0, a0        # t0保存返回值
17
18  lw a0, 4(sp)
19  lw ra, 0(sp)
20  addi sp, sp, 8    # 恢复并清空栈
21
22  mul a0, a0, t0    # a0: k * factorial(k - 1)
23  j end
24
25  base_case:
26      li a0, 1
27
28  end:
29  ret
30
31  ##### main #####
32  .data
33  n: .word 10
34
35  .text
36  main:
37      lw a0, n
38      call factorial

```

RISC-V 调用约定

RISC-V 调用约定 (Calling Convention)

Registers	Symbolic names	Description	Saver
x0	zero	Hardwired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporary registers	Caller
x8-x9	s0-s1	Saved registers	Callee
x10-x11	a0-a1	Function arguments and return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporary registers	Caller

- caller: 调用者保存 (绿框框起来的)
- callee: 被调用者保存